



Anticipating to Change: A Proactive Approach for Concept Drift Adaptation in Data Streams

Juan Valentín Guerrero Cano¹ · Gabriel Jonas Aguiar¹ · Alberto Cano²

Received: 10 June 2025 / Revised: 27 August 2025 / Accepted: 2 December 2025 /

Published online: 14 December 2025

© The Author(s) 2025

Abstract

Adapting to drifting data streams remains a key challenge in online learning, where effective model adaptation depends on timely concept drift detection. Most existing approaches respond to drift only after distributional changes occur, reacting to concept drift, limiting their ability to prevent the classifier's performance degradation. This work introduces a novel methodology to anticipate concept drift and enable proactive adaptation before the data distribution shift negatively impacts the classifier. We propose four proactive adaptation strategies based on the Very Fast Decision Tree (VFDT) algorithm to leverage data trends to estimate proactive changes in the classifier, mitigating or even preventing the performance degradation consequence of the concept drift. We evaluate the proposed methods across four scenarios with diverse data stream configurations. Results demonstrate that proactive adaptation reduces the adverse effects of concept drift and improves classification performance. In particular, the proposed strategies consistently outperformed in settings with incremental drift, underscoring the potential of anticipatory approaches and addressing a notable gap in the current literature.

Keywords Machine learning · Data streams · Concept drift · Proactive learning

Editor: Joao Gama.

✉ Juan Valentín Guerrero Cano
guerrerocajv@vcu.edu

Gabriel Jonas Aguiar
aguiargj@vcu.edu

Alberto Cano
acano@vt.edu

¹ Department of Computer Science, Virginia Commonwealth University, Richmond, VA, USA

² Department of Computer Science, Virginia Tech, Blacksburg, VA, USA

1 Introduction

The astonishing velocity at which data is generated nowadays has substantially increased over the past decades. The information generated by worldwide communications, social media, the Internet of Things, and other innovative technologies is often handled through continuous frameworks, due to its potentially infinite character (Chen and Zhang, 2014; Isah et al., 2019). Traditional machine learning models, which are mostly trained in static and batch data, struggle to learn from continuous and evolving information sources, commonly referred to as streams. The static assumptions on which these models are based, prevent them from effectively capturing the information these dynamic data streams provide.

The principal feature of data streams is their evolving nature, as the probabilistic distribution underlying the continuous arriving information may change over time. This phenomenon, known as concept drift, poses one of the main challenges in the field of data streams (Agrahari and Singh, 2022; Khamassi et al., 2018). Concept drift can significantly affect model predictions, making the learned knowledge obsolete. Research community has proposed several methodologies to address the challenges induced by concept drift. The most prevalent approach involves detection mechanisms (Agrahari and Singh, 2022; Ditzler et al., 2015), which rely on triggering signals to adjust the model to the detected concept drift. Conversely, implicit mechanisms do not wait for a specific signal, but naturally understand the underlying change on the data distribution concept drifts cause.

However, traditional approaches only considered reacting to concept drift rather than anticipating and early adapting to it (Gama et al., 2014). Although these methods offer robust and optimal results for most concept drifts scenarios, they still present opportunities for improvement. By merely reacting to concept drift, the models require significant enough evidence to confirm changes in the data distribution before adjusting their parameters accordingly. Consequently, this evidence often leads to performance degradation, as for a certain period of the data stream, the model is obsolete, relying on outdated knowledge. In this context, estimating future data distribution trends may help to anticipate, or at least mitigate, the impact of concept drift.

This work proposes a formal definition of such future estimation to mitigate and anticipate concept drift, which we define as *proactive model adjustment*. The main contributions of this paper are:

- Definition of proactive model adaptation: We comprehensively define reactive and proactive model adaptation to categorize how methods address concept drifts. This categorization offers a novel and powerful perspective on managing concept drift proactively and its potential benefits.
- Proactive version of VFDT: We propose a proactive version of the Very Fast Decision Tree (VFDT) by leveraging different proactive adaptation strategies to modify VFDT architecture as an example of the advantages of proactive models.
- Experimental comparison between proactive and reactive models: We conduct a study to analyze how different proactive strategies can perform across various concept drifts, benchmarking against a natural reactive baseline, VFDT.
- Recommendations and open challenges: We provide a comprehensive analysis of the advantages and limitations of proactive approaches, offering suggestions to build proactive adaptation strategies. We also outline potential research directions for future inves-

tigations into proactivity in data streams.

This paper is organized as follows: Sect. 2 provides the foundational knowledge necessary to understand the current approaches in data stream, discussing various methodologies for handling concept drift. Section 3 presents the definition of reactive and proactive model adjustments. Section 4 details the methodology of the empirical study, explaining in detail the proactive strategies compared. Section 5 describes the experimental setup and evaluation metrics. Section 6 presents and discusses the experimental results. Finally, Sects. 7 and 8 provide a comprehensive summary of the work, including our recommendations and future research challenges.

2 Background and Related Work

Data streams are potentially infinite sequences of data instances that arrive over time, often with high velocity. Formally, we define a data stream $\mathcal{D}$ as an infinite sequence of subsets $\mathcal{D} = \{S_1, S_2, \dots, S_\infty\}$ where each subset S_i is received at time i . Depending on the learning setting, each subset S_i may consist of a single instance (referred to as online data stream learning) or multiple instances (a batch data stream learning scenario). In supervised data stream learning, each instance is of the form (x, y) , where y is a label. In unsupervised data stream learning, the labels are absent (Agrahari and Singh, 2022; Lu et al., 2018; Aguiar et al., 2024).

Each subset of the data stream is drawn from a time-dependent probability distribution, denoted as D_t , where t represents discrete time steps. In a stationary data stream, this distribution remains constant over the time; otherwise, the stream is considered non-stationary. Specifically, D_t represents the joint distribution of the random variables X (feature space) and Y (label space), i.e. $D_t \sim \mathbb{P}_t(X, Y)$. In the context of data streams, D_t is referred to as the concept at time t .

Most real-world data streams are non-stationary, meaning the underlying distribution can change over time. Formally, there may exist some $t^* \in \mathbb{N} : D_{t^*} \neq D_{t^*+1}$. This phenomenon is known as concept drifts and stands as one of the main challenges in data stream learning (Agrahari and Singh, 2022; Gama et al., 2014; Aguiar and Cano, 2024). From a probabilistic perspective, D_t denotes the distribution at time t . By the chain rule, we can factorize the joint distribution as:

$$\mathbb{P}_t(X, Y) = \mathbb{P}_t(X) \mathbb{P}_t(Y | X) \quad (1)$$

Although this notation follows a discrete-style convention, both X and Y can be discrete or continuous in practice. The same decomposition applies in the continuous case, with $\mathbb{P}_t(\cdot)$ interpreted as a probability density function.

Concept drift can be categorized according to several characteristics, including its impact on decision boundaries, the speed of change, its locality, and its recurrence pattern (Agrahari and Singh, 2022; Aguiar et al., 2024):

Influence of the decision boundaries. Focuses on how concept drift affects the classifier's decision boundaries. Specifically, we can distinguish between drifts that alter posterior probabilities versus those that only affect the feature distribution. *Real concept drift*,

that occurs when the posterior probability $P(Y = y_i | X)$ changes, which directly shifts the decision boundaries. This typically leads to performance degradation unless the model adapts accordingly. *Virtual concept drift*, which occurs when the distribution of the input features $P(X)$ changes while $P(Y = y_i | X)$ remains constant. In such cases, although the distribution of features evolves, the decision boundaries may remain effective, since the underlying feature-label relationship is preserved.

Locality. As defined by Khamassi et al. (2018), Aguiar and Cano (2024), locality considers two aspects: whether the concept drift affects a single class or multiple classes, and whether the drift impacts only a portion of a class distribution (*local*) or the entire class (*global*). Based on these aspects, four types of drift can be identified: *Single-class Local* concept drift, when only one class is partially affected by the drift; *Single-class Global* concept drift where the entire distribution of one class is affected; *Multi-Class Local* concept drift, when multiple classes are each partially affected; and *Multi-Class Global* concept drift, where the whole distribution of multiple classes are impacted by the concept drift.

Recurrence. This characteristic describes whether previously observed concepts reappear over time. Recurrence can be classified as: *Acyclic*, when there is no specific periodicity for the recurrence of a previous concept, resulting in uncertainty about when concept may reappear; and *Cyclic*, when the recurrence of previous concepts follows a predictable pattern or periodic cycle, indicating when concept drift is likely to occur again.

As a consequence of the concept drift, the knowledge acquired by a model may become outdated, leading to increased error rates and reduced performance. To address this challenge, it is essential to implement adaptation mechanisms that allow the model to adjust in response to distribution changes, and maintain stable performance over time.

In the literature, several categorizations of adaptation mechanisms have been proposed, often sharing similar ideas with slight variations in interpretation. Common distinctions include active vs. passive approaches (Ditzler et al., 2015; Elwell and Polikar, 2011) where active methods rely on explicit drift detectors to trigger adaptation, while passive ones continuously update under the assumption that change may occur. Another widely used classification is blind vs. informed adaptation (Khamassi et al., 2018; Gama et al., 2014; Gomes et al., 2017a), where blind strategies adjust the model without detecting drift, whereas informed ones require a trigger. More recently, a distinction that has gained attention differentiates between explicit and implicit adaptation mechanisms (Aguiar et al., 2024): Explicit adaptation depends on external detectors that signal when to adapt, while implicit adaptation relies on mechanisms embedded in the classifier itself. These categorizations are not mutually exclusive, and their strong conceptual overlap can lead to ambiguity when trying to differentiate them. In this work, we focus on the explicit versus implicit distinction, as it provides a structured and clear framework for analyzing how models handle concept drift.

Implicit Adaptation. Implicit adaptation relies on the internal update mechanism of the learning algorithm to continuously adjust model structure or parameters without requiring an explicit drift signal. These methods typically employ incremental updating strategies that adapt with every new instance to changes in the underlying distribution. A representative example is the Concept-Adapting Very Fast Decision Tree (CVFDT)(Hulten et al., 2001). CVFDT extends the original VFDT algorithm by introducing a sliding window mechanism to handle concept drift. As new instances arrive, older ones are gradually forgotten, enabling the model to focus on the most recent data. Internally, CVFDT maintains alternate subtrees for parts of the model where recent data suggest a different splitting criterion might perform

better. When an alternate subtree demonstrates superior performance over time, it replaces the original subtree, allowing the model to adapt to evolving concepts without requiring an explicit drift signal.

Explicit Adaptation. Explicit methods depend on external mechanisms that detect concept drift and trigger model adaptation. Typically, a detection system signals when distributional changes occur, prompting an associated adaptation strategy. These approaches are especially valuable when drift characterization is required (Khamassi et al., 2018).

As highlighted in Ditzler et al. (2015), Lu et al. (2018), a broad range of detectors and strategies fall under this category. Most of these techniques follow a structured framework: data retrieval, data modeling, compute test statistics, and apply hypothesis tests to determine drift (Lu et al., 2018). These detectors are commonly grouped as follows.

- **Error rate-based detectors** monitor performance metrics such as classification error, raising alarms when statistically significant deviations are observed. For example, Drift Detection Method (DDM) (Gama et al., 2004) tracks the error over a sliding window and triggers adaptation when degradation crosses the warning and alarm thresholds. ADaptive WINdowing (ADWIN) (Bifet and Gavaldà, 2007) compares statistics across dynamic window segments and reduces its size when significant changes are detected. These methods are typically supervised, as they rely on labeled data to measure performance.
- **Data distribution-based detectors** assess changes in the underlying data distribution directly, without relying on model performance. They apply dissimilarity metrics and hypothesis testing between historical and incoming data to detect drift in the distribution (Lu et al., 2018). Notable examples include Statistical Change Detection for multi-dimension data (SCD) (Song et al., 2007), Competence Model-based drift detection (CM) (Lu et al., 2016), PCA-based Change Detection framework (PCA-CD) (Qahtan et al., 2015) and Least Squares Density Difference-based Change Detection Test (LSDD-CDT) (Bu et al., 2016). These are often referred to as unsupervised detectors.
- **Multiple hypothesis testing** frameworks combine several tests either in parallel (e.g., Just-In-Time adaptive classifiers (JIT) (Alippi and Roveri, 2008), Linear Four Rate drift detection (LFR) (Wang and Abraham, 2015), Information Value and Jaccard similarity (IV-Jac) (Zhang et al., 2017)) or hierarchically (e.g., Hierarchical Change-Detection Tests (HCDTs) (Alippi et al., 2016), Hierarchical Linear Four Rate (HLFR) (Yu and Abraham, 2017) or Two-Stage Shift-Detection Exponentially Weighted Moving Average (TSMDS-EWMA) (Raza et al., 2015)). These methodologies increase robustness and reduce false positives by integrating multiple evidence sources.

Concept drift adaptation, also referred to as drift reaction, encompasses methods designed to update the model in response to detected changes in the data distribution, ensuring continued predictive accuracy. According to Lu et al. (2018), these adaptation strategies can be grouped into three main categories:

- **Simple retraining:** When drift is detected, the model is retrained from scratch using recent data. For example, Paired Learners (Bach and Maloof, 2008) maintains two models: a stable learner optimized for long-term performance and a reactive learner tuned to short-term changes. When the reactive learner consistently outperforms the stable one,

it replaces it, thus adapting to the new concept.

- **Ensemble retraining:** Instead of discarding all past models, ensemble methods preserve and combine models trained across different concepts. Upon drift detection, a new model is trained and added to the ensemble, often replacing outdated ones. Leveraging Bagging (Bifet et al., 2010), combines bagging with ADWIN-based drift detection to replace underperforming models. Adaptive Random Forest (ARF) (Gomes et al., 2017b) extends the traditional random forest algorithm by using ADWIN to determine when outdated trees should be replaced.
- **Model adjusting:** Rather than retraining from scratch, some algorithms incrementally adjust internal components to adapt. As mentioned before, CVFDT (Hulten et al., 2001) extends VFDT by incorporating alternate subtrees and a sliding window to adapt structure over time. The Hoeffding Adaptive Tree (HAT) (Bifet and Gavalda, 2009) represents a more sophisticated evolution of the VFDT family, designed to handle concept drift through localized adaptation mechanisms. HAT incorporates ADWIN drift detectors at each internal node to continuously monitor the quality of splitting attributes by tracking the accuracy of predictions made by the subtree rooted at that node. When a drift detector signals a significant change in performance, HAT identifies the affected node and adaptively prunes the corresponding subtree, replacing it with a new one trained on recent data. This node-level adaptation allows HAT to respond to localized concept changes while preserving stable parts of the tree structure, making it particularly effective for handling drift that affects only specific regions of the feature space.

While both implicit and explicit methods offer viable solutions for handling concept drift, each comes with its own advantages and limitations. The selection of an approach depends on factors such as drift speed, computational constraints, and adaptation requirements. Below, we summarize the main challenges associated with these methodologies.

Limitations of Implicit Methods. Implicit methods are generally less popular than explicit ones due to their limitations, primarily their slow adaptation to concept drift. Since they update the model at a constant rate, regardless of the drift characteristics (e.g., abrupt or high-speed gradual drift), they struggle with sudden changes. Moreover, they assume a continuously evolving environment, leading to frequent model updates that may be computationally expensive or inefficient, especially when dealing with data streams where samples arrive rapidly or the stream remains mostly stationary. Methods such as VFDT rely on statistical indicators to update; however, this often results in error accumulation and delayed adaptation. Despite these drawbacks, implicit methods tend to perform well under gradual and low-speed drifts. Their effectiveness also depends on careful parameter tuning to balance stability and sensitivity (Gama et al., 2014), requiring a tradeoff to optimize performance in different drift scenarios.

Limitations of Explicit Methods. Explicit methods rely on triggering adaptation upon drift detection, but their effectiveness depends heavily on the chosen detection strategy. Most triggers are based on model error rates or data distribution changes, both prone to issues. A key challenge is the trade-off between false alarms and missed detections: The former can lead to unnecessary model updates and increased overhead, while the latter delay adaptation and harm performance. Therefore, explicit methods require careful calibration to balance sensitivity and specificity (Khamassi et al., 2018).

Therefore, both explicit and implicit methods can be regarded as reactive by nature, although in different ways. Explicit methods wait for a detection signal before adapting, while implicit methods update continuously based on accumulated evidence (e.g., recent performance or distributional changes). In both cases, adaptation occurs only after the drift has manifested, rather than in anticipation of it.

Several authors have explicitly classified these methodologies as reactive approaches (Khamassi et al., 2018; Ditzler et al., 2015; Han et al., 2022). Gama et al. (2014) argue that concept drift is naturally unpredictable, making its detection and handling predominantly reactive. Similarly, Aguiar et al. (2024) highlight this as an open challenge, noting that current adaptation techniques, particularly for imbalanced data and concept drift scenarios, still depend on changes in the data distribution having already occurred before adapting.

In contrast to reactive approaches, several studies have explored “proactive” strategies for handling concept drift. However, most of these works focus mainly on anticipating drift through detection mechanisms (drift prediction) rather than directly adapting the model before drift manifests (Gunasekara et al., 2024; Escovedo et al., 2018). One of the earliest proactive approaches in data stream learning was proposed by Yang et al. (2006), who introduced RePro, a hybrid framework that combines reactive and proactive methodologies to handle recurring concept drift. RePro’s reactive component builds a history of observed concepts through standard drift detection and continuously monitors for recurring patterns in previously encountered concepts. Its proactive component learns transition patterns between concepts using Markov Chains, allowing the model to anticipate and prepare for future drifts when similar patterns reappear. However, RePro is specifically constrained to scenarios with clear recurring patterns and requires sufficient historical data to build reliable transition models.

Similarly, Reactive and Proactive Concept Drift detection Ensemble (RePCoDE) (Ang et al., 2010) addresses concept drift in P2P networks by combining reactive and proactive adaptation strategies. RePCoDE’s proactive component identifies peers whose historical drift patterns resemble the local peer’s current trend through sequence similarity matching. Then, it selects and adapts models from these peers that represent post-drift concepts as predictors of future data. This enables proactive model adaptation before the local drift fully manifests, though the approach is limited to P2P environments and requires coordination between distributed peers, restricting its applicability to centralized learning scenarios.

More recently, Chen et al. (2016) introduced ProSeed, which represents a shift toward volatility pattern-based proactive approaches. ProSeed enhances drift detection by incorporating historical drift rate information through probabilistic networks that model stream volatility patterns. Building upon Seed drift detector (Huang et al., 2014), ProSeed transforms reactive volatility detection into a proactive prediction mechanism. Instead of merely reacting to detected drift, it learns transitions between volatility patterns and drift detections to predict the timing and location of future drifts. By modeling these volatility trends as pattern transitions in a probabilistic network, ProSeed adapts its data compression strategy to focus detection efforts on predicted drift periods. Building on similar principles, ProChange (Koh et al., 2018) extends this volatility-based approach to transactional data streams, using probabilistic networks to predict drift locations and dynamically adapt confidence thresholds for both real and virtual concept drifts in unlabeled data. However, both approaches are specifically constrained to streams with recurring volatility patterns and require accurate volatility

shift detection to build reliable predictive models, limiting their effectiveness in streams with other characteristics.

Detect Abrupt Drift (DetectA) (Escovedo et al., 2018) takes a different approach by using unsupervised clustering to enable proactive drift detection. The method applies k-means clustering to unlabeled test data and statistical hypothesis tests before true labels arrive. DetectA's proactive capability allows it to anticipate drift and either retrain the classifier with clustered labels or apply pattern mean shift corrections to incoming data. However, DetectA is specifically designed for abrupt drift scenarios, where sudden distributional changes make its statistical tests and pattern correction mechanisms more effective.

These detection-focused proactive methods have paved the way for more sophisticated adaptation strategies. Nacre (Wu et al., 2021) addresses the challenge of reducing performance loss between drift appearance and detection by leveraging concept recurrences in data streams. The framework proposes a novel drift coordinator that anticipates the next drift point and assesses the incoming concept to enable smooth adaptation. Nacre consists of three main components: a recurrent concept drift classifier, PEARL (Wu et al., 2020), which detects drift and models concepts, a drift sequence predictor that trains on sequences of drift intervals to predict future drift points, and the drift coordinator that orchestrates their interaction to perform both proactive drift detection and online updates. The key innovation lies in using drift sequences to anticipate future changes, allowing the system to be more proactive rather than purely reactive. However, Nacre assumes the existence of patterns in drift intervals and concept recurrence, requiring availability of labeled data and sufficient historical drift sequences to build reliable prediction models.

Beyond the classification paradigm, proactive adaptation has also been applied to data stream processing tasks. For instance, Stream Window Join (SWJ) operations, which involve joining data streams with disordered arrival times (also referred to as stream oscillation), present significant challenges. To address this, Proactive Error Compensation-Join (PECJ) (Zeng et al., 2024) was proposed as a proactive solution that accounts for anticipated future contributions from delayed data, thereby improving join accuracy.

In the domain of time series forecasting, Proceed (PROactive model aDaptation) (Zhao and Shen, 2024) represents one of the most recent advancements in proactive drift adaptation. Unlike ProSeed, which focuses on drift detection, Proceed is designed for proactive adaptation, aligning closely with the objectives of our work. Proceed addresses delays in obtaining ground-truth labels, which can lead to models adapting to past and outdated concepts rather than current distributions. The key idea behind Proceed is that changes in the data distribution correspond to modifications in model parameters, meaning that concept drift can be anticipated by learning a mapping from the concept space to the model's parameter space. By combining this novel perspective with drift estimation techniques, Proceed enables models to proactively adjust their parameters before the new concept fully manifests.

Furthermore, Zhao and Shen (2024) introduces a formal definition of online model adaptation, which aligns with the definitions of proactivity and reactivity discussed in the next section. Accordingly, we expand upon this definition to provide a more rigorous and generalized framework for understanding proactive and reactive adaptation in data stream learning.

3 Reactive and Proactive Concept Drift Adaptation

The terms **proactivity** and **reactivity** are commonly used to describe behavioral patterns in living beings. Proactivity refers to the ability to anticipate future events and take preventive actions to either avoid potential problems or make informed decisions in advance, ensuring preparation and mitigating possible consequences. Conversely, reactivity describes behavior in which actions are taken only after a problem has already arisen, responding to its consequences rather than anticipating them. Both terms are fundamentally related to time and anticipation, as reactive behavior addresses problems after they have materialized, whereas proactive behavior seeks to prevent them.

In the context of data streams, these concepts have been adopted to characterize different approaches to handling concept drift. The term reactive has been used to describe methodologies that respond to concept drift only after they have already manifested (Khamassi et al., 2018; Ditzler et al., 2015; Han et al., 2022). Similarly, the term proactive has been used for methodologies that attempt to anticipate concept drift or that continuously update model parameters (Gama et al., 2014; Gomes et al., 2017a). However, despite their widespread use in the literature, there exists no formal definition for these concepts in the concept drift adaptation domain.

To address this gap, we propose formal mathematical definitions for both reactivity and proactivity in concept drift adaptation. These definitions serve as fundamental criteria for distinguishing between adaptation paradigms and provide a clearer framework for understanding how methods handle distributional changes.

Before defining reactive and proactive adaptation strategies, we must establish a fundamental criterion that distinguishes between these paradigms. This criterion is based on the concept of model outdatedness.

A model $\mathcal{M}$ is considered **outdated** at a given time step t if it has not yet adapted to an ongoing concept drift. In other words, the model parameters at time t , denoted as Θ_t , remain aligned with a previous data distribution D_{k-1} rather than the current distribution D_k .

Such outdatedness manifests as a significant performance degradation in predictive accuracy, which can be quantified using an error function $\mathcal{L}(\cdot)$. Formally, if concept drift is present from time t' onward, the model is considered outdated if the following condition is satisfied:

$$|\mathcal{L}(\mathcal{M}_{t'}, S_{t'}) - \mathcal{L}(\mathcal{M}_{t'-1}, S_{t'-1})| \geq \epsilon \quad (2)$$

where ϵ is a sensitivity threshold, $\mathcal{M}_{t'}$ and $\mathcal{M}_{t'-1}$ represent the models with parameters $\Theta_{t'}$ and $\Theta_{t'-1}$ respectively, and $S_{t'}$ and $S_{t'-1}$ denote sets of instances drawn from distributions $D_{k'}$ and $D_{k'-1}$ respectively. Note that we deliberately distinguish indices for instance arrival time steps (t') from indices for concept distributions (k'), as these may not coincide in practice.

This definition of outdatedness is crucial because it establishes the baseline condition that separates reactive approaches (which allow models to become outdated) from proactive approaches (which aim to prevent outdatedness).

3.1 Reactive Adaptation

We can define a model $\mathcal{M}$ as **reactive** if its adaptation mechanism is designed to update the model only after it has become outdated due to concept drift. In other words, it remains outdated for a certain period after concept drift starts, only adapting once its outdated state is detected. Reactive models fundamentally operate on a *problem-detection paradigm*: they identify when current performance or structure is suboptimal and then adapt accordingly.

Formally, given that concept drift starts at t' , a reactive adaptation mechanism is triggered at time $t' + K$, with $K \in \mathbb{N}^+$, upon detecting the outdated state of the model, leading to an adjustment to recover from it:

$$\mathcal{M}_{t'+K} = \mathcal{A}_{re}(\mathcal{M}_{t'+K-1}, S_{t'+K}), \quad (3)$$

where $\mathcal{A}_{re}$ represents the reactive adaptation mechanism.

Reactive models exhibit a common behavior: they wait until outdatedness is recognized before adjusting. This paradigm encompasses both explicit and implicit adaptation approaches from traditional taxonomies, with some exceptions, as both fundamentally respond to manifested problems rather than prevent them.

Most explicit approaches (excluding those using proactive detectors such as DetectA or ProSeed) rely on external drift detection mechanisms like DDM or ADWIN that signal when distribution changes occur, subsequently triggering model adaptation. These methods separate the detection phase from the adaptation response, operating with a delay between the onset of concept drift and the recognition of model outdatedness. This makes most explicit approaches reactive, as they can only adapt after the drift has manifested and been detected. This reactive nature is particularly evident in error-based detectors, which explicitly require performance degradation to occur before signaling adaptation needs. For example, ensemble methods like Adaptive Random Forest use dedicated detectors to identify when outdated trees should be replaced.

Most implicit approaches integrate detection and adaptation within the learning process itself. CVFDT exemplifies this reactive behavior by monitoring statistical confidence through alternate subtrees and replacing tree structures only when these alternatives demonstrate superior performance on recent data; it can only adapt after sufficient evidence of current structure inadequacy has been accumulated. Similarly, VFDT exhibits reactive learning behavior by waiting for sufficient statistical evidence before making structural decisions. Figure 1 illustrates this during an incremental concept drift, where VFDT remains outdated during the transition period, leading to observable performance degradation until sufficient evidence is collected to trigger adaptation by replacing the corresponding leaf node with a new split node. It is important to note that VFDT does not possess an adaptation mechanism for concept drift but exhibits a reactive learning process that naturally makes its behavior reactive to distributional changes.

The defining characteristic that unites all reactive models is that they respond to detected problems rather than prevent them. Whether through external detectors or integrated learning mechanisms, they do not anticipate distributional changes but instead attempt to recover from outdatedness after concept drift has already affected their performance.

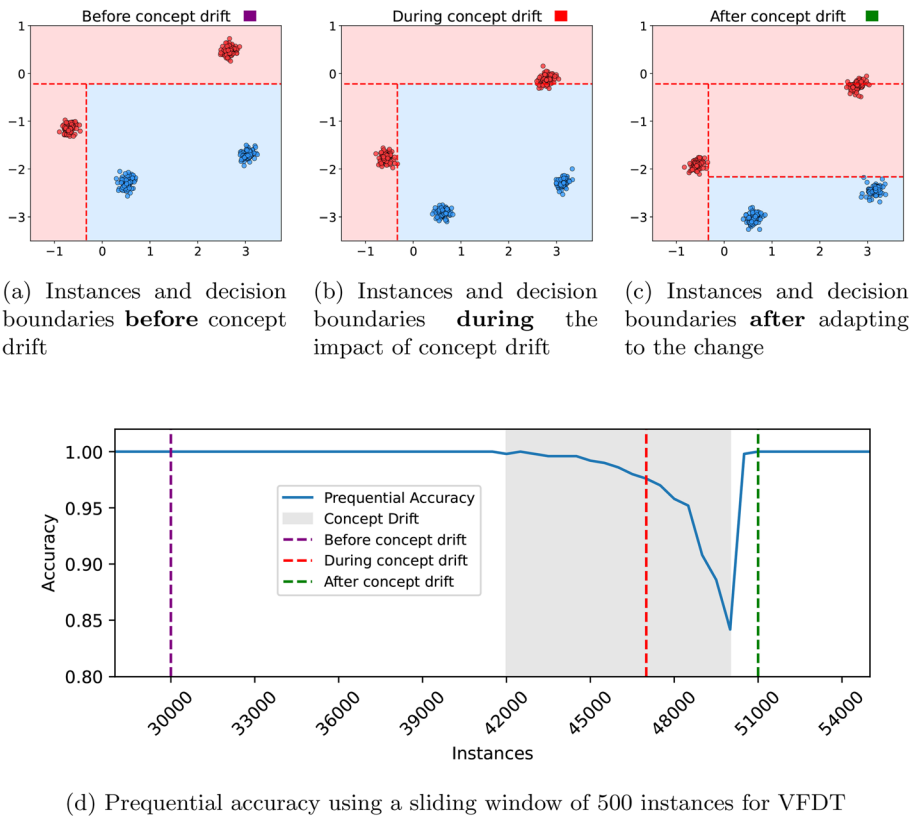


Fig. 1 Example of reactive learning process of VFDT during an incremental concept drift. **a** shows the data stream and decision boundaries before the concept drift occurs; **b** illustrates the ongoing concept drift; **c** shows the updated decision boundaries and data stream after the concept drift; and **d** shows the prequential accuracy of VFDT over the data stream. VFDT only reacts to the concept drift after enough evidence is collected, replacing the corresponding leaf node by a new split node. Consequently, a performance degradation can be observed in **(b)** until VFDT reacts to the change **(c)**

3.2 Proactive Adaptation

A model $\mathcal{M}$ is **proactive** if its adaptation mechanism is designed to mitigate or avoid the negative performance consequences of an incoming concept drift, i.e., prevent a future outdated state. Proactive models fundamentally operate on a *prevention paradigm*: they anticipate potential distribution changes and adjust their parameters accordingly before becoming outdated.

Formally, given a concept drift starting at time t' , a proactive model performs an adaptation step at $t' - \alpha \geq 0$:

$$\mathcal{M}_{t'-\alpha} = \mathcal{A}_{pro}(\mathcal{M}_{t'-\alpha-1}, S_{t'-\alpha}). \quad (4)$$

where $\mathcal{A}_{pro}$ represents the proactive adaptation mechanism that aims to estimate the proper adjustments on the model to avoid an outdated state. Thus, the model attempts to meet the condition:

$$|\mathcal{L}(\mathcal{M}_{t'}, S_{t'}) - \mathcal{L}(\mathcal{M}_{t'-1}, S_{t'-1})| < \epsilon. \quad (5)$$

when the concept drift starts at t' .

Unlike reactive models, proactive models do not rely on detecting outdatedness but rather anticipate potential changes and adjust accordingly. This anticipation is non-deterministic, in the sense that it relies on estimating future data trends rather than reacting to observed changes.

The adaptation mechanism $\mathcal{A}$ is the fundamental component that determines whether adaptation is reactive or proactive. Strictly speaking, our categorization depends on the nature of this adaptation mechanism. In practice, a model may employ multiple adaptation components with different characteristics, some reactive and others proactive. By convention, we classify a model as proactive when at least one of its adaptation mechanisms operates proactively.

Proactive methods can leverage various sources of information for anticipation, including historical concept patterns (e.g., RePro using Markov chains), current distributional trends (e.g., Proceed mapping distribution changes to parameter space), or even structural patterns in incoming unlabeled data (e.g., DetectA). Regardless of the information source, the key characteristic is that these methods attempt to adapt before performance degradation occurs.

Proactive adaptation strategies can be broadly categorized into two main approaches:

1. **Algorithm-level proactive adaptation:** These methods modify internal model parameters or structure directly based on anticipated changes. For example, Proceed proactively modifies model parameters by learning a mapping from distributional changes to the parameter space, enabling direct parameter adjustment before new concepts fully manifest.
2. **Data-level proactive adaptation:** These methods leverage instances from the stream itself to guide the learning process, allowing the classifier to anticipate upcoming changes in the data distribution. This includes techniques such as generating synthetic instances that simulate future conditions, reusing old samples from previous concepts, or selectively reweighting recent observations to guide the model toward anticipated changes.

Figure 2 illustrates the behavior of proactive adaptation during an incremental concept drift, where the model anticipates distributional changes and maintains stable performance by adapting decision boundaries before significant performance degradation occurs.

3.3 Drift Type Considerations and Limitations

The effectiveness of proactive adaptation is strongly influenced by the characteristics of concept drift. Certain types of concept drifts, such as virtual concept drifts, do not require any adaptation as they do not influence the decision boundaries learned by the classifier.

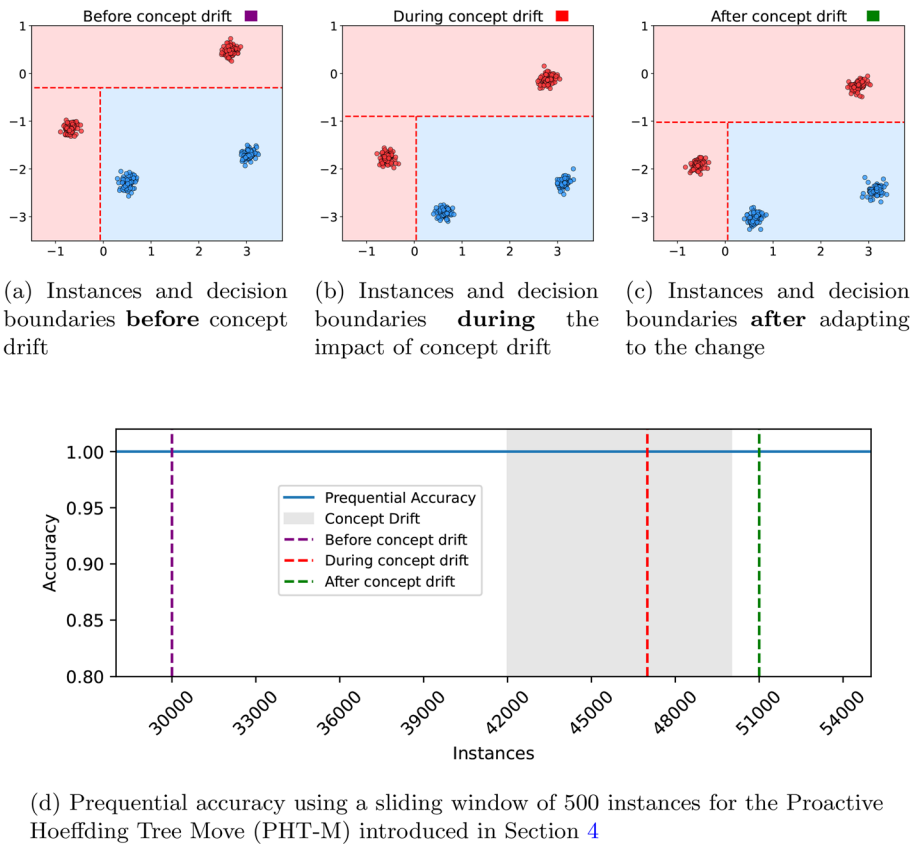


Fig. 2 Example of proactive adaptation of PHT-M (Section 4.1.1) during an incremental concept drift. **a** shows the data stream and decision boundaries before the concept drift occurs; **b** illustrates the ongoing concept drift; **c** shows the updated decision boundaries and data stream after the concept drift; and **d** presents the prequential accuracy of VFDT over the data stream. The threshold is adapted based on the most recent window of 1000 instances, maximizing class separability and allowing anticipation of clusters movements

This means that no degradation in performance is observed, which, in terms of our definitions, implies that even if a virtual concept drift occurs, the model remains up-to-date.

When considering different drift patterns, acyclic abrupt concept drifts present unique challenges since there is no feasible way to anticipate them through historical patterns. In contrast, abrupt but cyclic concept drifts may exhibit trends in the history of concept drifts that allow predicting their occurrence, enabling proactive adaptation. However, when dealing with acyclic and abrupt drifts, models must rely primarily on their reactive adaptation components. It is notable that DetectA can proactively detect abrupt concept drifts, but this is achieved by working with unlabeled input data before their ground-truth labels arrive rather than predicting future drift manifestation.

While proactive adaptation aims to prevent performance degradation before outdatedness occurs, its effectiveness depends critically on the accuracy of future estimations. In cases where drift is highly unpredictable or erratic, proactive adjustments may fail to align

with actual distribution changes, potentially leading to suboptimal model configurations. Thus, the success of proactive methods is strongly influenced by the complexity and predictability of the concept drift patterns in the data stream (Gama et al., 2014; Lu et al., 2018; Escovedo et al., 2018).

4 Proposed Proactive Approaches

In this section we present four proactive approaches that are based on different proactive adaptation strategies. In order to implement these variations, we used Very Fast Decision Tree (VFDT) (Domingos and Hulten, 2000) as our base classifier.

Domingos and Hulten introduced VFDT as one of the first algorithms for incrementally constructing decision trees from data streams under the assumption of a stationary distribution (Domingos and Hulten, 2000). Although VFDT by itself does not adapt to concept drift, it incrementally updates its structure as more data arrive, which can modify parts of the tree when the distribution changes (Gama et al., 2014). In practice, effective drift adaptation usually requires augmenting VFDT with explicit mechanisms such as sliding windows, adaptive pruning, or drift detectors (e.g., CVFDT, HAT). We selected VFDT as our base model due to its modular and interpretable architecture and its strong empirically proven performance, making it well suited to apply controlled model adjustments (Pfahring et al., 2007; Ikonomovska et al., 2011). Additionally, its simple but effective algorithm led to its use as a building block in numerous adaptive and ensemble-based models in the literature (Gomes et al., 2017a; Krawczyk et al., 2017).

This combination of robustness, reactivity, and flexibility makes VFDT an excellent candidate to evaluate the potential of different proactive strategies. For our experimentation, we used the implementation of VFDT in the River library (Montiel et al., 2021), keeping all default hyperparameters except for the leaf prediction mechanism, which was modified to majority class one to ensure visually consistent output results.

4.1 Proactive Hoeffding Tree

VFDT provides a strong reactive baseline, adapting only after concept drift has begun and the model's decision boundaries have already been affected. To explore the potential benefits of proactive adaptation, we introduce a set of proactive strategies applied to VFDT, collectively referred to as the Proactive Hoeffding Tree (PHT).

We propose four variations of PHT, each one incrementally incorporating proactive mechanisms. All variations share a window-based adaptation process, where adjustments are executed based on the most recent window of data, reinforcing the influence of recent observations to proactively anticipate concept drift (Lu et al., 2018). During our experimentation, we used a window size $|W|$ of 1000 samples, allowing proactive adjustments to be performed without excessive delay. This size was chosen considering the speed of the concept drifts studied in our experimentation.

Additionally, in all proactive strategies presented in the following subsections, we employ a consistent notion of distance to study the trajectories of data samples in relation to a given decision boundary learnt by the model. A decision boundary is defined by a feature-threshold pair (f, τ) , and the distance of a sample x to the boundary is computed as

the absolute difference $|x_f - \tau|$. Each sample is assigned to one of the two regions induced by the boundary ($x_f \leq \tau$ or $x_f > \tau$). We identify the *closest points* as the samples with the smallest distances in each region, subject to a predefined per-region quota, ensuring a balanced selection of reference points on both sides of the boundary. This consistent distance definition is applied across all upcoming proactive approaches.

4.1.1 Proactive Hoeffding Tree Move

The concept of local adaptation in decision trees has been widely studied in the literature, mainly due to their flexible architecture, which allows for dynamic modifications at the node level (Lu et al., 2018; Ikonomovska et al., 2011). Proactive Hoeffding Tree Move (PHT-M) builds upon the VFDT, leveraging its malleable architecture, by introducing an enhanced yet simple mechanism for updating decision boundaries which aims to mitigate the impact of incremental concept drift by adjusting decision boundaries dynamically.

The mechanism adapts the threshold, which remains fixed in the standard VFDT, at split nodes by analyzing the most recent $|W|$ observations. This strategy can be classified as an algorithm-level proactive adaptation mechanism, as it modifies the representation that VFDT uses to define decision boundaries (feature-threshold pair). By dynamically adjusting these thresholds based on recent data samples, PHT-M anticipates and adapts to gradual concept drifts before classification performance significantly deteriorates. Figure 3 presents how PHT-M proactive adaptation mechanisms works in two dimensional case. For a given window of points, the centroid of the closest points in each region is computed. The decision threshold is then updated to the mean feature value of these centroids, allowing the model to anticipate to incremental concept drifts.

The threshold adaptation in PHT-M can be divided in the following steps. First, the algorithm maintains a sliding window of the most recent $|W|$ observations, where $|W|$ is a predefined parameter. Every $|W|$ samples the proactive adjustment is performed. Second, for each decision boundary, feature-threshold pair (f, t) in the decision tree, the mechanism selects the closest points from among those that, when traversing the tree, reach the leaves under the decision node where the boundary, (f, t) , is defined. That is, instead of considering all points in the current window, only those that fall into the corresponding leaves are eligible for selection. Next, the closest points are divided into two regions, separated by

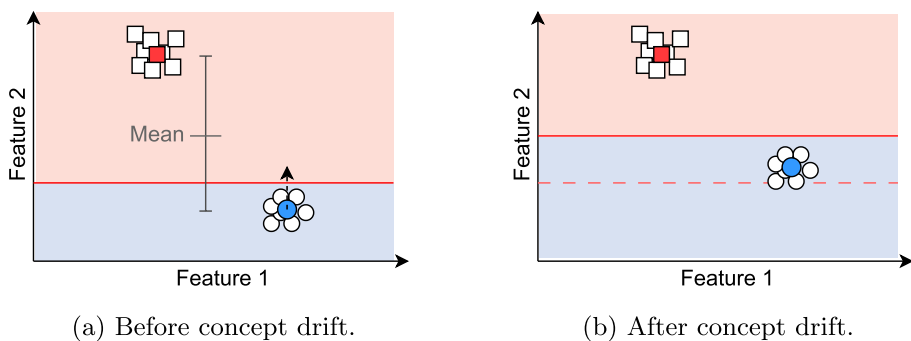


Fig. 3 Representation of proactive adaptation mechanism of PHT-M in two-dimensional case. **a** presents the data stream and decision boundaries before the concept drift and **b** after the concept drift. The dashed line in **b** represents the old decision boundary that would have become outdated

the threshold t : (R_1) , points whose feature value exceeds the threshold ($x_f > t$), and (R_2) points whose feature value is less or equal to the threshold ($x_f \leq t$). The size of R_1 and R_2 is determined by the hyperparameter α and the size of the window for proactive adaptations: $|R_i| = \alpha \cdot |W|$ for $i = 1, 2$. Lastly, the new threshold is calculated as the midpoint between the mean feature values of the two regions points.

$$t' = \frac{\frac{1}{|R_1|} \sum_{x \in R_1} x_f + \frac{1}{|R_2|} \sum_{x \in R_2} x_f}{2} \quad (6)$$

Then, the algorithm updates the threshold in the corresponding decision node, preserving the structure of the tree.

The proactive adaptation mechanism of PHT-M is illustrated in Algorithm 1.

Input: *Tree*, window W , quota factor α

```

foreach  $v \in \text{InternalNodes}(\text{Tree})$  do
     $(f, t) \leftarrow \text{DecisionBoundary}(v);$            // feature--threshold at  $v$ 
     $k \leftarrow \lfloor \alpha \cdot |W| \rfloor;$                      // per-region quote
     $(R_1, R_2) \leftarrow \text{GetFilteredClosestPoints}(W, v, f, t, k);$ 
     $\mu_1 \leftarrow \frac{1}{|R_1|} \sum_{x \in R_1} x_f;$ 
     $\mu_2 \leftarrow \frac{1}{|R_2|} \sum_{x \in R_2} x_f;$ 
     $t' \leftarrow (\mu_1 + \mu_2)/2;$ 
     $\text{UpdateThreshold}(\text{Tree}, v, t');$ 

```

Algorithm 1 PHT-M proactive adaptation mechanism

The adaptation mechanism described above enables PHT-M to proactively anticipate gradual concept drifts by adjusting decision boundaries based on the data distribution changes rather than reacting to performance degradation (consequence of the VFDT learning process). This characteristic directly aligns with the definition of proactive adaptation.

The proposed adaptation algorithm can also be interpreted as a margin maximization approach. In a binary classification problem, the ideal decision boundary should lie in an area of low data density, maximizing the margin between classes (Cao et al., 2006). By computing the centroid of points for the given feature on either side of the decision boundary, PHT-M approximates an optimal hyperplane between the two subset of closest points, R_1 and R_2 .

Focusing exclusively on points near the decision boundary provides an optimal balance between computational efficiency, adaptation relevance (prioritizing closest points as most informative samples), and stability (preventing drastic decision boundaries modifications due to outliers).

4.1.2 Proactive Hoeffding Tree Move and Rotate

Similarly to the previous proactive approach, Proactive Hoeffding Tree Move and Rotate (PHT-MR) extends VFDT by introducing an adapting mechanism not only for thresholds of the decision nodes, but also for the features used in split nodes. This allows PHT-MR to

rotate the feature-threshold hyperplane, redefining the decision boundaries in response to changes in data distribution.

In certain cases, when the distribution of points in the input space evolves due to incremental concept drift, a simple adjustment to the threshold of the split node is insufficient. The decision boundary may become misaligned with the optimal and intuitively expected decision hyperplane, what can be solved by modifying the orientation of the decision boundary.

Figure 4 illustrates the proactive adaptation mechanism of PHT-MR in the two-dimensional case, where a rotation of the decision boundary is executed. This rotation is performed according to the angle, θ , between the centroids of the closest points to the decision boundary in each region. If the angle is below a predefined threshold, the decision boundary is rotated to the feature selected by a heuristic score (excluding the current one). In 2D input space, the rotation is performed to the unique possible feature. VFDT waits until sufficient statistical evidence is collected to replace a leaf node by a split node, resulting in the creation of an entirely new decision boundary, even when a simple adjustment to the existing one could have sufficed. PHT-MR addresses this inefficiency by introducing a straightforward but effective technique to rotate decision boundaries whenever such situations can be estimated. This is achieved by analyzing the angle formed between the centroids of the closest points on either side of the decision boundary and the decision boundary itself.

The adaptation mechanism employed by PHT-MR can be summarized in several steps. Firstly, as PHT-M algorithm does, it maintains the sliding window and computes the closest points for each decision boundary in the tree. Right after, the closest points for each decision boundary are divided into two regions R_1 and R_2 just as in PHT-M. The centroids of both regions are computed, c_1 and c_2 as:

$$c_1 = \frac{1}{|R_1|} \sum_{x \in R_1} x, \quad c_2 = \frac{1}{|R_2|} \sum_{x \in R_2} x$$

The alignment between the centroid difference vector $\vec{v} = c_1 - c_2$ and the decision boundary's normal vector $\vec{n}$ is calculated:

$$\theta = \cos^{-1} \left(\frac{\vec{v} \cdot \vec{n}}{\|\vec{v}\| \|\vec{n}\|} \right)$$

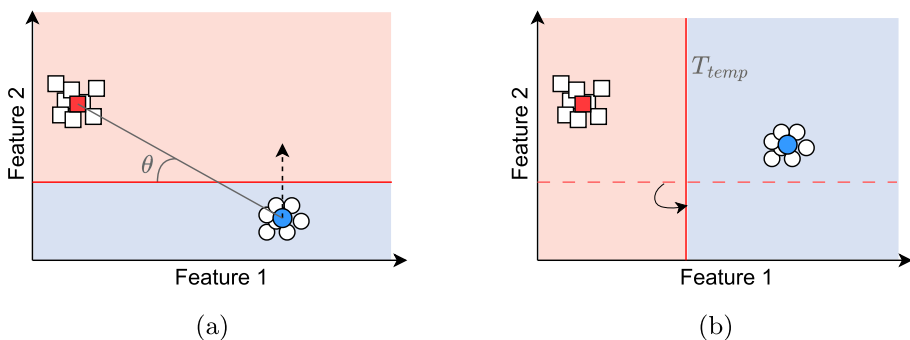


Fig. 4 Representation of proactive adaptation mechanism for rotation of PHT-MR in two-dimensional case. **a** presents the data stream and decision boundaries before the concept drift and **b** after the concept drift. The dashed line in **b** represents the old decision boundary that would have become outdated

If the angle θ is below a predefined threshold, Θ , rotation is applied. A low angle value indicates that $\vec{v}$ is nearly parallel to the decision boundary's normal, i.e., nearly orthogonal to the decision boundary. Alternatively, a high angle value suggests that $\vec{v}$ is almost parallel to the hyperplane. When a rotation is estimated to be necessary, a heuristic measure is used to select a new feature (excluding the current one) that allows for a better separation of the closest points. Finally, the new threshold is computed identically to the PHT-M approach, just taking in account the possible rotation made before.

Algorithm 2 illustrated the pseudocode for the proactive adaptation mechanism of the PHT-MR.

Input: *Tree*, window W , quota factor α , rotation threshold Θ

foreach $v \in \text{InternalNodes}(\text{Tree})$ **do**

```

    ( $f, t$ )  $\leftarrow$  DecisionBoundary( $v$ );    // current feature--threshold
     $k \leftarrow \lfloor \alpha \cdot |W| \rfloor$ ;                // per-region quote
    ( $R_1, R_2$ )  $\leftarrow$  GetFilteredClosestPoints( $W, v, f, t, k$ );
     $c_1 \leftarrow$  ComputeCentroid( $R_1$ );        // in feature space
     $c_2 \leftarrow$  ComputeCentroid( $R_2$ );
     $\vec{v} \leftarrow c_1 - c_2$ ;
     $\vec{n} \leftarrow$  NormalVector( $\vec{v}$ );
     $\theta \leftarrow \cos^{-1} \left( \frac{\vec{v} \cdot \vec{n}}{\|\vec{v}\| \|\vec{n}\|} \right)$ ;
    if  $\theta > \Theta$  then
         $f' \leftarrow$  SelectNewFeature( $W, v, f$ );    // exclude  $f$ 
         $t_{temp} \leftarrow (c_1[f'] + c_2[f'])/2$ ;    // midpoint along  $f'$ 
         $rot = \text{True}$ 
        ( $R_1, R_2$ )  $\leftarrow$  GetFilteredClosestPoints( $W, v, f', t_{temp}, k$ )
         $\mu_1 \leftarrow \frac{1}{|R_1|} \sum_{x \in R_1} x f'$ ;   $\mu_2 \leftarrow \frac{1}{|R_2|} \sum_{x \in R_2} x f'$ ;
         $t' \leftarrow (\mu_1 + \mu_2)/2$ ;
        UpdateBoundary( $\text{Tree}, v, f', t', rot$ );

```

Algorithm 2 PHT-MR proactive adaptation mechanism

Following the same principles as PHT-M, this approach aims to position the decision boundary in an optimal region, maximizing the separation between the closest points on both sides. The rotation mechanism in PHT-MR further enhances its ability to anticipate complex scenarios where the current decision boundary becomes outdated.

Two key aspects of this approach are worth highlighting. First, the heuristic selected to determine the best feature for performing the rotation is the variance. Variance provides insight into which feature may offer the greatest separation between data points, although this might not always be true. The information gain heuristic, used by VFDT River's default implementation, was discarded because of the suboptimal performance obtained during the experimentation. However, these heuristics can be adapted or replaced depending on the characteristics of the stream analyzed. Second, when a rotation is performed, a temporal threshold is initially assigned to the newly selected feature. This is necessary because the threshold of the previous feature is no longer valid. Therefore, a temporary estimation is made until sufficient observations allow for a more stable and well-adjusted threshold value. Since the rotation condition is based on the angle between the centroids and the decision

boundary being below a certain threshold, the temporal threshold is estimated as the mean value of the previous computed centroids along the new feature.

PHT-M and PHT-MR operate on a prevention paradigm rather than problem-detection. Unlike CVFDT, which waits for alternate subtrees to demonstrate superior performance before replacement, PHT-M and PHT-MR continuously optimize decision boundaries every $|W|$ samples without waiting for performance degradation. The key distinction lies in their trigger mechanism: CVFDT responds to detected inadequacy by comparing competing structures, while PHT-M and PHT-MR proactively maximize class separability based on recent data patterns to prevent future outdatedness. This continuous optimization for robustness, rather than reactive replacement after detecting problems, exemplifies the prevention paradigm that defines proactive adaptation.

4.1.3 Proactive Hoeffding Tree Move and Create

Proactive Hoeffding Tree Move and Create (PHT-MC) incorporates the same proactive threshold adaptation mechanism as PHT-M but extends it to anticipate more complex situations. This approach estimates whether a leaf node should be replaced by a subtree to anticipate a possible concept drift that could affect the decision boundary its parent split node represents. The subtree is constructed manually, following specific strategies to ensure consistency and mitigate the consequences of the incremental concept drift. Figure 5 illustrates an example in which the creation of a new subtree would be a suitable solution. Figure 5a represents a complex situation in which the red class trajectory is estimated to collide with the current decision boundary represented in the tree of Fig. 5c. Figure 5b illustrate the solution PHT-MC proposes to tackle the problem, which can be seen as creating the subtree represented by Fig. 5d.

The reactive baseline, VFDT, would not be able to anticipate the situation described in Fig. 5. Initially, it would create a new decision boundary within the current blue region but would misclassify instances until sufficient evidence of the red class accumulates in that region. Only then it would create a second new decision boundary that would finally classify adequately. In contrast, PHT-MC aims to address this issue by constructing the two-level subtree in a single step, by estimating the trajectory of the gradual concept drift, and analyzing the distribution of points within the blue region. A more detailed explanation of the adaptation mechanism of PHT-MC is provided below, along with the pseudocode in Algorithm 3 which offers a high level overview of it.

After applying the same threshold adaptation mechanism that PHT-M uses, the novelty of PHT-MC is introduced. At every $|W|$ samples, for each leaf in the tree, its history is updated. The history of a leaf consists of past centroids of the closest points to its parent's decision boundary that were classified by the leaf. This history information is updated with the most recent window of observations. Then, by using the accumulated history information for each leaf, the algorithm estimates whether the centroid of the samples classified by a leaf in the next window will move too close to the decision boundary. This means a potential situation for performance degradation may occur. Right after, if a potential performance degradation is detected, the approach builds a complete subtree to replace the sibling node of the leaf. Note that PHT-MC replace the sibling node of the one that is being analyzed, as the performance degradation will be suffered in that sibling leaf node of the tree due to the concept drift (points are moving towards the region classified by the sibling leaf node).

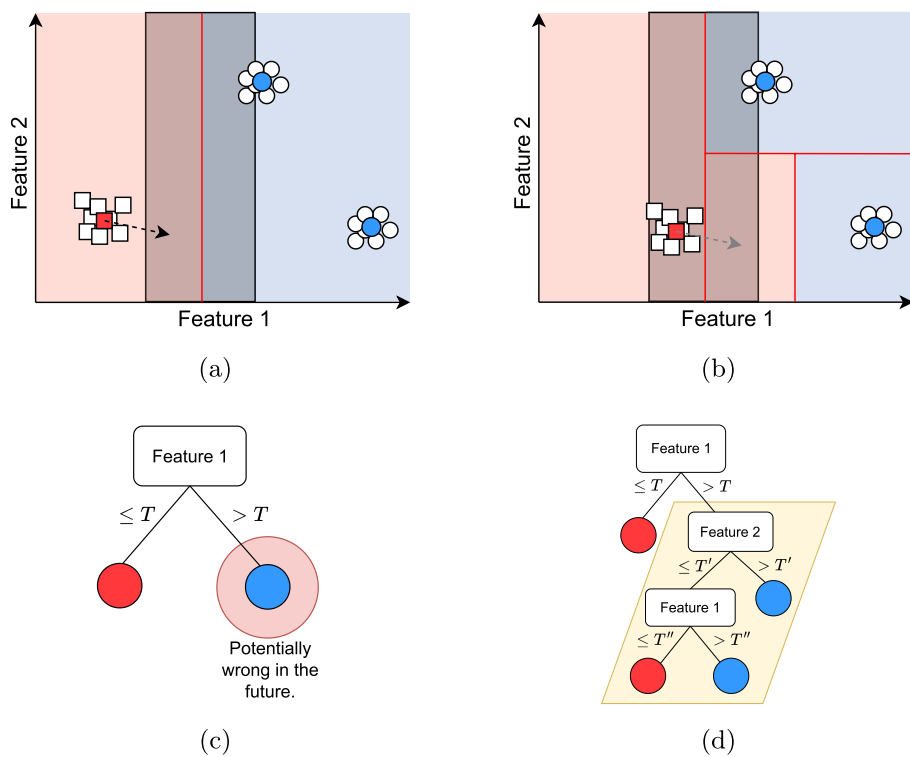


Fig. 5 Representation of proactive adaptation mechanism for creating a subtree of PHT-MC in two-dimensional case. **a** presents the data stream and decision boundaries before the concept drift and **b** after the concept drift. Respectively, **c-d** illustrates the decision tree before and after the concept drift. The yellow background represent the subtree created by PHT-MC to replace the leaf node

To determine the appropriate structure of the subtree, PHT-MC analyzes the relationship between the centroid of the closest points to the parent's decision boundary and all the window samples classified by the sibling leaf. Based on this analysis, PHT-MC decides whether to create a 1-level or 2-level subtree mainly depending on the data distribution in the potentially invaded region.

- **1-level subtree:** The first step involves creating a new decision boundary in the region predicted to be invaded. This decision boundary is positioned at the midpoint between the centroids of the closest points of both regions, considering the feature that best separates them. Figure 6 provides visual illustration of this case. The feature selection heuristic is used to determine which feature is most appropriate to define the new split node.
- **2-level subtree:** After applying the first split, PHT-MC further analyzes the subregion on the side of the new boundary expected to be invaded. If there are signs of potential degradation even after the first split (existing points in this new subregion) an additional split is performed. This results in a 2-level subtree, which allows some room for the new points in the potentially invaded region. Figure 5 illustrates the process for a 2-dimensional input space.

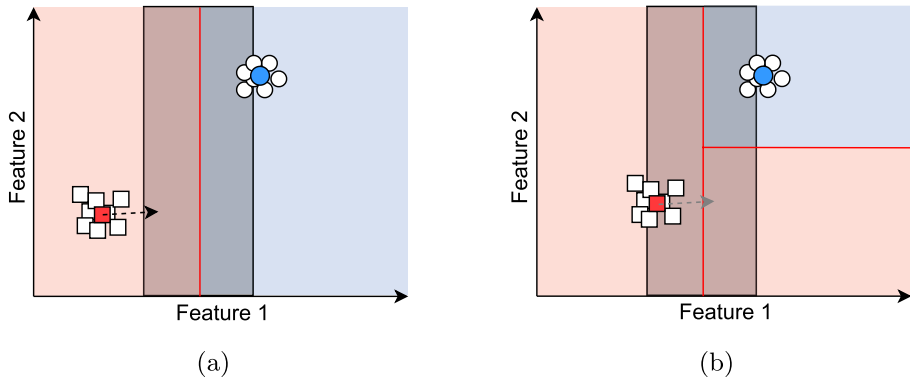


Fig. 6 Only one additional split (1-level subtree) is needed for the PHT-MC to anticipate to the estimated movement of the red class. **a** presents the data stream and decision boundaries before the concept drift and **b** after the concept drift. The gray dashed line in **b** represents the estimation of the next centroid of closest points

Input: *Tree*, window W , quota factor α , distance threshold τ

$k \leftarrow \lfloor \alpha \cdot |W| \rfloor$;

$T' \leftarrow \emptyset$;

foreach $v \in \text{InternalNodes}(Tree)$ **do**

$(f, t) \leftarrow \text{DecisionBoundary}(v)$;

$(R_1, R_2) \leftarrow \text{GetClosestPointsPerLeaf}(W, v, f, t, k)$;

$\mu_1 \leftarrow \frac{1}{|R_1|} \sum_{x \in R_1} x f$; $\mu_2 \leftarrow \frac{1}{|R_2|} \sum_{x \in R_2} x f$;

$t_v \leftarrow (\mu_1 + \mu_2)/2$;

$\text{UpdateThreshold}(Tree, v, t_v)$;

$T'[v] \leftarrow (f, t_v)$;

$L \leftarrow \text{UpdateLeafHistories}(Tree, W)$;

foreach $\ell \in L$ **do**

$v \leftarrow \text{Parent}(\ell)$; $s \leftarrow \text{Sibling}(\ell)$;

$(f, t_v) \leftarrow T'[v]$;

$m \leftarrow \text{CentroidShift}(\ell, W, f)$;

if $|m - t_v| \leq \tau$ **then**

$f' \leftarrow \text{SelectFeature}(Tree, v, s, W)$;

$t' \leftarrow \text{ComputeNewThreshold}(W, f')$;

$T \leftarrow \text{CreateProactiveSubtree}(Tree, v, s, f', t')$;

if $\text{RegionAtRisk}(T, W)$ **then**

$f'' \leftarrow \text{SelectFeature}(T)$;

$t'' \leftarrow \text{ThresholdByCentroids}(W, f'')$;

$\text{ExtendSubtree}(T, f'', t'')$;

$\text{ReplaceSibling}(v, s, T)$;

Algorithm 3 PHT-MC proactive adaptation mechanism

Several of PHT-MC hyperparameters were manually selected based on empirical results. As a result, while this approach may not always fully anticipate the concept drift, it effectively mitigates performance degradation in most of the cases studied. A more refined tuning of these hyperparameters for specific scenarios could potentially lead to improve performance.

To decide the feature to create the split on when creating the subtree, PHT-MC uses the same heuristic measure as PHT-MR does, the variance along each feature for the given points. This allow us to select the feature where the points seems more separable.

In addition, the creation of a deeper subtree (2-level subtree) is particularly effective in 2-dimensional spaces, where a second split may adequately divide the points. However, in higher-dimensional spaces, additional splits may be required to fully adapt to the changing data distribution. Despite this, the proposed technique still mitigates performance degradation by improving the separability of different classes within the affected region.

4.1.4 Proactive Hoeffding Tree Shadow

Proactive Hoeffding Tree Shadow (PHT-S) is the last proactive adaptation mechanism introduced to modify the standard VFDT, aiming to anticipate to gradual concept drifts. This approach is presented last due to its distinct nature; while previous approaches focused on modifying parameters learned by the VFDT (algorithm-level adaptation), PHT-S rely solidly on VFDT's mechanism while allowing some degree of proactivity.

Unlike other approaches, PHT-S does not update the threshold or the feature of node splits. Instead, its adaptation mechanism is similar to that used by PHT-MC when creating a new subtree to replace existing leaf nodes. However, the key difference between PHT-S and PHT-MC lies in how these subtrees are created. Instead of manually constructing the subtree based on observed samples, PHT-S trains a new subtree (called shadow tree) by using semi-synthetic data points.

This proactive training process occurs when the proximity of the data points to a decision boundary suggests an imminent change. In such cases, a shadow tree is trained using critical points from the potentially invaded region and synthetic ones, which are generated by estimating the future movement of the data points. This estimation is based on a history of previous distribution's states, which allows the algorithm to simulate the likely structure of future windows. Once the estimated trajectory suggests that the new data will cross the decision boundary in the next iteration of the proactive mechanism, the shadow tree replaces the potentially invaded leaf node, mitigating the performance consequences of the concept drift.

The algorithm maintains a sliding window of the most recent W observations, where W is a predefined parameter. Every W samples, the proactive adjustment is performed. Then, at each iteration, the history information is updated by storing previous states of the sample distribution. The history allows the model to estimate future trends based on recent changes. After updating the history, the algorithm checks whether the centroids of the closest points to a decision boundary (for a given split node) have entered a danger region, defined by the threshold movement (τ) hyperparameter. As in PHT-MC, if any centroid falls within this region, it is considered a potential risk for the current decision boundary, and a "shadow" tree (Hoeffding Tree classifier) is created if it has not been previously initialized. The shadow tree is designed to replace the leaf node that represents the region predicted to be invaded. If the shadow tree already exists for a given leaf node, it is incrementally trained

with all the samples from the current window that have been classified by that leaf. Additionally, the algorithm identifies invader points, which are the closest points to the decision boundary that might cross it in the near future. These invader points are artificially shifted using an adjustment vector, computed based on the history information collected, illustrated in Fig. 7. The shadow tree is then trained using this adjusted dataset, combining the invader points with the current samples classified by the leaf. Finally, when the model estimates that the points will cross the decision boundary in the next data window, the leaf predicted to be invaded is replaced by the trained shadow tree.

Algorithm 4 provides a high-level overview of the adaptation mechanism PHT-S implements.

Several techniques that generate synthetic data points to specialize model training have already been described in the literature (Aguilar et al., 2024; Li et al., 2022). This data-level approach enables us to leverage VFDT's reactive mechanism in a proactive way without modifying its parameters. Instead, we identify critical points near the decision boundary and estimate their future positions, focusing on training the shadow tree to anticipate future changes in the data distribution. When the situation is critical, the shadow tree is used to mitigate the performance degradation that occurs when gradual concept drifts affect decision boundaries.

Unlike the other proactive strategies described above, PHT-S minimizes reliance on heuristics. The only estimation performed is to generate synthetic points, resulting in a more natural and compatible adaptation mechanism with VFDT.

Input: *Tree*, window W , quota factor α , distance threshold τ

```

 $k \leftarrow \lfloor \alpha \cdot |W| \rfloor$ ;
foreach  $v \in \text{InternalNodes}(\text{Tree})$  do
     $(f, t) \leftarrow \text{DecisionBoundary}(v)$ ;
     $(R_1, R_2) \leftarrow \text{GetFilteredClosestPoints}(W, v, f, t, k)$ ;
     $L \leftarrow \text{UpdateLeafHistories}(\text{Tree}, v, R_1, R_2)$ ;
    foreach  $\ell \in L$  do
         $c \leftarrow \text{Centroid}(\ell, R_1, R_2, f)$ ;
        if  $|c - t| \leq \tau$  then
            if  $\ell$  not in Danger region then
                 $\text{InitShadow}(\ell) \leftarrow \text{HoeffdingTreeClassifier}()$ 
                 $\text{Synt} \leftarrow \text{ShiftPoints}(\ell, R_1, R_2)$ ;
                 $P \leftarrow \text{Synt} \cup \text{LeafPoints}(\text{Sibling}(\ell), W)$ ;
                 $\text{Train}(\text{Shadow}(\ell), P)$ 
            if  $\text{InvasionEstimated}(\ell, f, t, W)$  and  $\text{HasShadow}(\ell)$  then
                 $\text{ReplaceSibling}(\text{Parent}(\ell), \text{Sibling}(\ell), \text{Shadow}(\ell))$ ;

```

Algorithm 4 PHT-S proactive adaptation mechanism

4.2 Linear Trajectory Assumptions and Method Constraints

The proposed PHT methods are fundamentally constrained by the linear nature of decision boundary adjustments in Hoeffding Trees. While the algorithms analyze the closest points

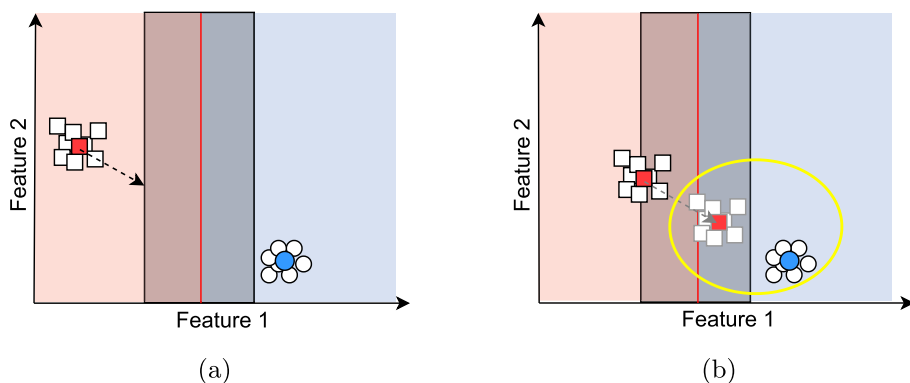


Fig. 7 Representation of points used for the training of shadow trees. **a** presents the data stream and decision boundaries before the concept drift and **b** after the concept drift. When the red class enters a critical region, light gray points are synthetically generated based on past trends. These synthetic points, along with the points in the blue region (yellow circle) are used to train the shadow tree

to decision boundaries to anticipate distributional changes, the available transformations for axis-aligned hyperplanes are linear: threshold movement (PHT-M), rotation to different features (PHT-MR), or creation of sub-hyperplanes (PHT-MC/PHT-S). These linear transformations can effectively accommodate distributional changes that align with geometric patterns, even when the underlying cluster movements are curved, as the hyperplane adjustments can still provide adequate approximations. However, the proposed methods may struggle when concept drift involves non-geometric patterns or overly complex geometric changes that cannot be adequately approximated through simple linear boundary transformations. The fundamental limitation emerges when distributional changes are either non-geometric in nature or too complex to be effectively approximated by linear hyperplane adjustments.

As mentioned in Sect. 2, various proactive algorithms exist that do not assume linear adjustments since they are not based on decision trees. Methods such as RePro or Proceed, could be more effective proactive strategies when dealing with drift conditions that do not represent geometric patterns suitable for linear hyperplane transformations.

5 Experimental Setup

The experimental study¹ aimed to assess the efficacy of the proposed proactive approaches in various conditions and levels of complexity, compared to an established classification algorithm in the literature, VFDT. The objective is to discern the contexts in which the proposed proactive mechanism outperforms existing state-of-the-art techniques, as well as contexts where its limitations become apparent. We will address the following research questions (RQ):

- **RQ1:** How do different PHT approaches compare to its standard reactive version,

¹All the code necessary to reproduce the experimentation described in this study is publicly available on Github (<https://github.com/Vgueca/ProactiveHT>).

VFDT, to anticipate and mitigate the effects of concept drift?

- **RQ2:** Under what conditions does proactive adaptation provide the greatest advantages over reactive adaptation?
- **RQ3:** How do different proactive strategies compare in terms of efficiency in anticipating concept drift?
- **RQ4:** Do proactive approaches maintain their advantages when applied to real-world binary classification datasets with natural concept drift patterns?
- **RQ5:** What is the impact of applying the proposed proactive mechanisms to other widely used tree-based classifiers?

5.1 Benchmarks

To systematically evaluate the performance of proactive adaptation mechanism, we designed a set of controlled experimental scenarios, each simulating different *incremental concept drifts*.

In order to build each scenario we have made use of the public code from Aguiar and Cano (2024), where the authors created synthetic concept drifts (both local and global) by using River library (Montiel et al., 2021). Specifically, the streams have been created using the *random radial basis* function generator provided by River, as other authors used this generator to build incremental concept drifts over synthetic data streams (Gomes et al., 2017b). Additionally, all generated streams are strongly class-balanced, ensuring that our analysis focuses exclusively on handling the concept drift, without introducing other factors.

We generated multiple experimental configurations by varying key characteristics that influence the complexity of the data stream. The number of classes was fixed at 2 to narrow the scope of the experimentation to binary class problems. Specifically, we varied the dimensionality of the feature space, the number of cluster per class, and the ratio of clusters affected by the drift, thus, obtaining a diverse range of streams configurations.

The selected parameters exposed in Table 1 cover a range of complexities, from low-dimensional settings with fewer clusters of samples to high-dimensional cases with complex structures. These variations allow us to analyze how proactive approaches perform under different degrees of data stream complexity. In addition, different random seeds were used to introduce variability and evaluate the robustness of the approaches, reinforcing the validity of the experimentation. Table 1 summarizes the parameter used to construct the 72 distinct configurations to build the base data streams before applying the drifts to them.

In each of the possible configurations shown in Table 1 we introduced four types of incremental concept drifts, each representing different cluster movements. These particular drifts were selected because they encompass the most common transformations observed in incremental concept drift. Table 2 provides a detailed description of each drift type, including the number of affected clusters.

Figure 8 provides an illustration of the different incremental concept drifts we used for our experimentation.

Parameters of the Data Stream. Each data stream consisted of 50,000 samples. However, in the *Chase* drift scenario, the number of samples may vary between 50,000 and 300,000 samples, as each movement along the drift path required 50,000 samples to be

completed. The total number of movements in the path was randomly determined, leading to variations in stream length for this drift type.

Drift Injection and Data Stream Speed. Each stream introduces concept drift through parameterized movement functions built upon the Random RBF generator (Aguiar and Cano, 2024). The scenarios define the type of cluster movement, the affected class proportions, and the drift dynamics. Most drifts extend over 50,000 samples, allowing for smooth and progressive transitions. In *Chase*, between 1 and 6 sequential drifts occur, each lasting 50,000 samples, yielding extended streams with long-term evolution. This scenario is particularly interesting to proactive strategies, as it offers prolonged exposure to gradual changes, where reactive approaches may suffer from continuous performance degradations.

Clusters are initialized randomly based on the seed, ensuring reproducibility across runs while preserving variability across configurations. The degree of overlap during drift varies: in *Cross*, clusters from different classes move toward each other and cross paths, generating high overlap and ambiguous regions. In contrast, *Merge* causes clusters belonging to the same class to converge toward a common centroid, occasionally aligning their movement with clusters from other classes and creating overlapping directional trends. These structural differences influence the amount and quality of proactive signals that models can exploit.

The speed of drift significantly affects proactive adaptation. Gradual changes provide enough samples to estimate trajectories and adjust thresholds, while faster shifts reduce the available information per region. In real-world high-speed streaming environments, fixed-size windows may become outdated rapidly, requiring more frequent updates with smaller proactive windows. However, smaller windows reduce the quality of trajectory information, limiting the reliability of proactive estimations. Thus, under extremely fast- or short-horizon drift, proactive models may underperform unless adaptive windows or hybrid strategies are employed.

To assess the proactive adaptation mechanism under different drift speeds, we built three accelerated versions of the original dataset. In the original setup ($\alpha = 1$), concept drifts last for the entire stream, creating slow incremental changes. To test scenarios with faster concept drift, we added an acceleration parameter $\alpha \in \{0.25, 0.50, 0.75, 1\}$, where lower α values mean shorter drift periods. When α decreases, the concept drifts take less time to complete, resulting in faster changes that need fewer samples compared to the original dataset. Through this systematic variation in drift speed, we aim to evaluate the robustness of our proactive adaptation mechanism under conditions where incremental concept drift occurs more abruptly, potentially challenging the algorithm's ability to anticipate and adapt to changes.

Sudden and Recurring Drift. The proactive mechanisms proposed in this study are specifically designed to operate under incremental concept drift, where changes occur progressively and can be anticipated by analyzing recent data trends in the stream. In contrast, sudden or abrupt drifts involve immediate and unpredictable shifts, typically offering no

Table 1 Configuration parameters used for generating the synthetic scenarios

Parameter	Values
Random Seed (s)	{2, 3, 4}
Dimensionality (N)	{2, 5, 10}
Clusters per Class (k)	{1, 2, 3, 4}
Number of Classes (C)	2
Affected Clusters Ratio (r)	{0.5, 1.0}

Table 2 Types of incremental concept drift applied to the data streams in our experimental scenarios. The cross drift requires an even number of clusters affected while the Merge requires at least two clusters per class

Drift type	Description
Chase	Clusters move sequentially along a randomly generated path, following movement vectors whose number and direction are randomly determined
Cross	Pairs of clusters swap positions by moving along vectors of equal magnitude but opposite direction
Split	A cluster divides into n smaller subclusters, where n is randomly determined (between 1 and 6). The resulting clusters follow independent movement trajectories
Merge	Pairs of clusters of the same class gradually merge by moving toward their centroid until they fully overlap. At least two clusters are required for this transformation

early indicators. Such behavior is incompatible with the assumptions underlying proactive strategies, and in these scenarios, reactive methods or drift detectors are more appropriate.

To evaluate the limitations of our proactive adaptation mechanism under challenging conditions, we designed a set of 10 synthetic streams of 10,000 samples each. These streams simulate the Chase scenario, generating between 4 and 6 random incremental concept drifts that are completed within only 250 samples each. This creates a particularly challenging evaluation scenario where concept drifts occur much more rapidly than in the standard experimental setup, allowing us to assess how our proposed models perform when abrupt concept drift is present.

Recurring drifts, on the other hand, may enable anticipation in certain cyclic contexts, provided that the model is equipped with mechanisms to recognize and exploit historical patterns. This, however, falls beyond the scope of our current work, which does not exactly incorporate memory of past concepts. Notably, RePro (Yang et al., 2006) is a hybrid framework that integrates reactive and proactive components and leverages a history of past drifts to anticipate future recurrences. Exploring similar hybrid strategies that combine memory-based and proactive adaptation remains an interesting direction for future research.

Real-World Data Stream. Additionally, to evaluate the proposed approaches under real-world conditions, we selected 7 binary classification data streams that represent scenarios where concept drift occurs in complex and unpredictable ways. The dataset was obtained from established data streams repositories, including the River library (Montiel et al., 2021) and the benchmark collection by Souza et al. (2020). These real-world streams provide an important test beyond our controlled synthetic experiments, as they include the noise, complexity, and irregular drift patterns found in real applications. The selected datasets (see Table 3) are commonly used benchmarks in concept drift research, allowing us to compare our results with existing studies.

5.2 Evaluation Metrics

We compared the models using the prequential accuracy metric, as different authors support prequential metrics for evaluating online learning models (Gama et al., 2013, 2009). Moreover, due to the balanced nature of the generated data streams only accuracy is presented in this work, alternative metrics such as precision, recall, or F1 score are provided in the code. To evaluate the approaches performance, we calculated the prequential accuracy using a rolling window of 500 instances. The accuracy at each step is computed as the proportion of correctly classified instances within the window, described by Eq. 7.

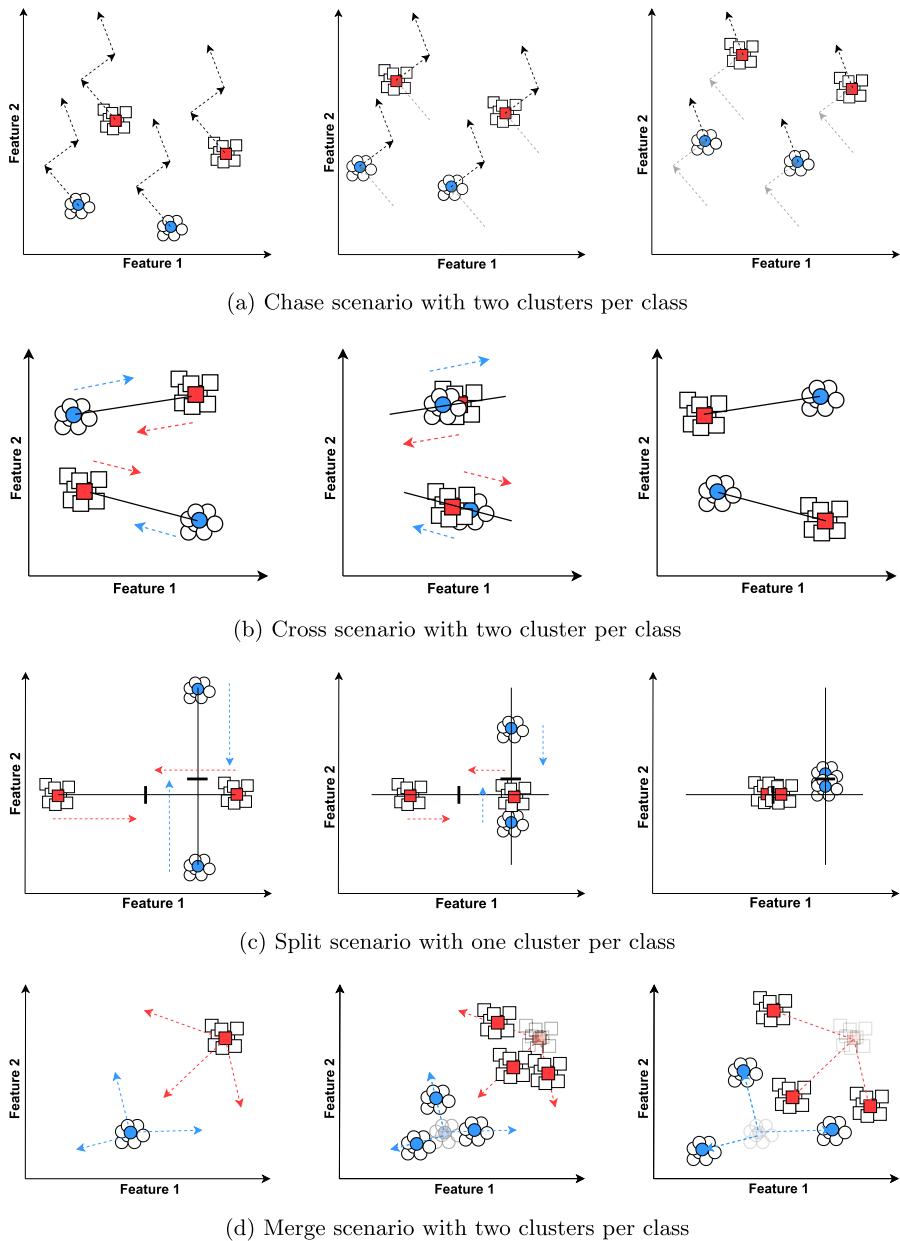


Fig. 8 Representation of the different scenario types in two-dimensional space. **a** illustrates Chase scenario; **b** Cross; **c** Split; and **d** Merge. The dashed lines represent the specific movement each cluster have to perform. In *Chase* scenario the movement is the same for all the clusters of points

$$\text{Pre-Acc}_t = \frac{\sum_{i=t-w}^t \mathbf{1}(\hat{y}_i = y_i)}{w} \quad (7)$$

where t denotes the time step corresponding to the arrival of instance t , w represents the rolling window size, y_i the true label, and $\hat{y}_i$ is the predicted label for instance i . This method provides a smoothed estimation of model performance while mitigating short-term fluctuations.

Inspired by previous work on concept drift adaptation (Gama et al., 2014; Shaker and Hüllermeier, 2015), we introduce three additional metrics to analyze how each strategy specifically manages the complexity of the experimental configurations. These metrics focus on study the adaptability and anticipation of the models:

- **Percentage of degradations episodes (PDE):** This metric counts how many distinct low-performance episodes the model experiences throughout the data stream. A low-performance episode is considered a drop on the prequential accuracy (relative to previous measurements) that exceeds an adaptive threshold (see Eq. 8). Fewer degradation episodes suggest a stronger capacity to anticipate or mitigate concepts changes. As the number of windows may vary across scenarios, we report the number of low-performance episodes per every 100, allowing direct comparison between distinct scenarios.

$$\theta_j = \alpha \cdot \frac{1}{w} \sum_{k=j-w}^{j-1} \text{Acc}_k, \quad (8)$$

where $\alpha \in (0, 1)$ (in our case, $\alpha = 0.95$) and w is the number of recent windows used to compute the baseline (in our case, $w = 5$). For ($j \leq w$), we set $\theta_j = \alpha \cdot \text{Acc}_j$, therefore not taking in count the first five windows (2500 samples) for this metric.

- **Fraction of time in degradation episodes (FTDE):** Measures the percentage of time (in terms of evaluation windows) during which the model's prequential accuracy remains below the threshold. Formally, it is obtained by dividing the number of windows the model is under a degradation episode by the total number of windows in the stream. Lower values indicate that the model has spent less time under suboptimal performance, reflecting a robust adaptation process in the presence of drift.

$$\text{Fraction Time Degraded} = \frac{1}{W} \sum_{j=1}^W \mathbf{1}(\text{Acc}_j < \theta_j). \quad (9)$$

Table 3 Real-world binary classification datasets used for evaluation

Dataset	Instances	Features	Classes
CreditCard	284,807	30	2
Electricity	45,312	8	2
NOAA Weather	18,159	8	2
MIRS	4,260	3,600	2
Yoga	3,300	426	2
Ozone	2,534	72	2
Phishing	1,250	9	2

where W denotes the number of windows of the given scenario.

- **Recovery time (RT):** Calculate the average number of windows required for the accuracy to rise back above the threshold once a degradation episode begins. Shorter average recovery times suggest faster adaptation to changes in data distribution.

$$\text{AvgRecoveryTime} = \frac{1}{K} \sum_{k=1}^K (e_k - s_k). \quad (10)$$

where K is the number of degradation episodes and s_k and e_k represent the start and end window of the degradation episode k .

Additionally, we monitored the memory consumption of each model over time. The memory footprint of each approach was recorded at intervals of 1000 instances (window size for proactive adjustments). This measurement includes the memory required to store decision tree structures, additional adaptation mechanisms, and any auxiliary data necessary for the drift anticipation.

By analyzing these metrics, we aim to provide a comprehensive evaluation of the trade-offs between proactive and reactive adaptation strategies, in terms of predictive performance, resource efficiency, and concept drift adaptation effectiveness. To ensure statistical significance of the comparisons (Bifet et al., 2015), we applied appropriate statistical tests based on the normality assumption of the differences in the compared metrics. Specifically, when normality was met, we used a paired t-test (parametric test), whereas in cases where normality was not satisfied, we applied the Wilcoxon Signed-Rank test to perform a robust non-parametric comparison (Bifet et al., 2015).

6 Results

This section presents and discusses the results obtained throughout the experimentation. First, we provide an integrated analysis that examines how the characteristics of the stream (dimensionality, number of clusters per class, and scenario types) affect the performance of the proposed proactive approaches. We then present a comprehensive overview of overall performance, including speed variation analysis, hyperparameter optimization, and computational efficiency. Finally, we evaluate the approaches on real-world data and assess the generalization capabilities of the proactive mechanisms to other popular tree-based classifiers.

6.1 Analysis by Stream Configuration

In this section, we present a comparative analysis of how each performance metric varies between different approaches for a given stream configuration. We consider these results to provide insightful information about how each approach behaves under different degrees of complexity in the streams.

Table 4 summarizes the metric results grouped by the dimensionality. In low-dimensional spaces, hyperplane-based models have fewer degrees of freedom to define effective decision

boundaries, limiting their ability to separate complex data distributions. Oppositely, higher-dimensional scenarios offer greater flexibility for defining such boundaries, thanks to the increased number of features. Moreover, high-dimensional settings are particularly relevant as they offer a better representation of real-world data streams.

In two-dimensional scenarios, the PHT-MC obtained the best results, with a prequential accuracy of 0.946 and an average of 0.851% of degraded episodes, which aligns with its proactive adaptation mechanism specially designed for two-dimensional cases. Interestingly, the results obtained by PHT-M and PHT-MR share strong similarities with those obtained by PHT-MC.

PHT-MC, an extension of PHT-M, is designed to slightly anticipate more complex situations than PHT-M. As shown in Table 4, this results in particularly favorable metrics in terms of adaptation to the concept drift. Compared to VFDT, PHT-MC experiences, on average, approximately 0.5% fewer degradation episodes (1.285 and 0.851 respectively). In practical terms, this implies that PHT-MC suffers less significant drops in accuracy than VFDT.

When considering higher-dimensional (5 and 10 dimensions) scenarios, PHT-MC does not achieve the best performance results, but closely follows the best approach, PHT-M. The latter significantly outperforms the reactive baseline, not only in terms of prequential accuracy but also in the proportion of degradation episodes. Specifically, VFDT suffers around twice as many degradation episodes as PHT-M under these settings.

PHT-MR has strong performance in two-dimensional scenarios (0.943 accuracy) because its rotation mechanism is stable when selecting from limited features. However, in higher dimensions, the feature selection heuristic becomes unreliable, causing performance to stay constant (≈ 0.942 across all dimensions) while other approaches improve with dimensionality. Similarly, PHT-S maintains competitive results but consistently underperforms compared to PHT-M and PHT-MC due to limitations in its synthetic data generation that could benefit from refinement.

In summary, the dimensionality of the scenario has a clear impact on performance and adaptation-related metrics. A consistent trend can be observed across all approaches: as the dimensionality increases, performance improves, as illustrated in Fig. 9. This improvement is likely due to the enhanced separability of the data in higher-dimensional spaces. However, some of the proactive approaches suffer from the complexity of their proactive adaptation mechanism, limiting its effectiveness in specific settings.

Similarly, Table 5 presents a detailed comparison of the results obtained by each approach as a function of the number of clusters per class, a hyperparameter that substantially influences the complexity of the scenarios. A lower number of clusters typically results in lower density of classes and, therefore, simpler separability between them. In contrast, a larger number of clusters significantly challenge the adaptation mechanisms of the approaches, as most diverse data regions may affect the decision boundaries. When the concept affects all the clusters in the data stream (i.e., $r = 1.0$), high-density settings can become particularly chaotic, making proactive anticipation more difficult.

In lower-complexity scenarios, such as those with one or two clusters per class, all proactive strategies outperform VFDT. Both, PHT-M and PHT-MC methods maximize the separability between the points on either side of each decision boundary. This strategy allows them to perform robustly regardless of the level of complexity the scenarios present. In many cases, simply repositioning the decision boundary to a more suitable location (mov-

Table 4 Aggregated comparison of approaches across different dimensions

Dimension	Model	Preq. Acc.	PDE (%)	FTDE	RT
2	VFDT	0.930 ± 0.066	1.285 ± 1.972	0.104 ± 0.148	5.276 ± 8.861
	PHT-M	0.943 ± 0.058	0.960 ± 1.622	0.081 ± 0.140	3.916 ± 6.854
	PHT-MR	0.943 ± 0.057	1.061 ± 1.689	0.072 ± 0.125	3.639 ± 6.685
	PHT-S	0.935 ± 0.062	1.937 ± 2.918	0.107 ± 0.135	3.830 ± 4.619
	PHT-MC	0.946 ± 0.054	0.851 ± 1.476	0.075 ± 0.131	3.910 ± 6.916
5	VFDT	0.943 ± 0.053	1.949 ± 2.164	0.102 ± 0.145	3.022 ± 3.702
	PHT-M	0.960 ± 0.038	1.027 ± 1.051	0.052 ± 0.104	2.586 ± 4.367
	PHT-MR	0.941 ± 0.043	1.732 ± 1.485	0.087 ± 0.116	4.136 ± 4.782
	PHT-S	0.948 ± 0.047	1.917 ± 2.163	0.098 ± 0.138	3.518 ± 4.404
	PHT-MC	0.958 ± 0.037	1.003 ± 1.032	0.057 ± 0.098	3.123 ± 4.622
10	VFDT	0.949 ± 0.057	1.147 ± 1.966	0.089 ± 0.162	2.969 ± 5.253
	PHT-M	0.969 ± 0.030	0.533 ± 1.114	0.040 ± 0.100	2.024 ± 4.383
	PHT-MR	0.942 ± 0.044	1.592 ± 1.785	0.093 ± 0.118	4.000 ± 4.243
	PHT-S	0.956 ± 0.046	1.147 ± 2.220	0.067 ± 0.125	1.941 ± 3.414
	PHT-MC	0.968 ± 0.029	0.480 ± 0.980	0.040 ± 0.087	2.537 ± 5.864

The best value for each metric is highlighted in bold

ing its threshold) is enough to significantly mitigate the accuracy drop consequence of the concept drift.

Interestingly, PHT-MR, which also builds upon PHT-M, achieved the best results when there is only one cluster per class (0.960 of accuracy). Its rotation mechanism relies on the closest points from both regions separated by the decision boundary to decide when and how to rotate it. In scenarios with lower number of clusters, the points in each region are

more likely to belong to the same cluster, resulting in more effective rotations. This may not hold when number of clusters increases, as shown Table 5, directly affecting the stability of the rotation.

As the scenario complexity increases, the results continue to favor PHT-MC and PHT-M, both achieving the highest prequential accuracy in the cases with three and four cluster per class, respectively. PHT-M exhibits slightly superior results in adaptation-related metrics. These findings align with the expected behavior of their adaptation mechanisms, which are highly robust to the different complexity levels of the scenarios. As anticipated, VFDT consistently struggles to adapt to the concept drift, presenting more frequent and prolonged degradation episodes.

Figure 10 provides a visual summary of metric trends across different cluster configurations. All approaches exhibit a consistent pattern: performance improves from one to two clusters per class, reaching a peak, then consistently decreases as cluster numbers increase. This trend suggests that the underlying VFDT benefits from moderate class variability but struggles with excessive complexity. The behavior can also be explained by the affected clusters ratio hyperparameter (r): scenarios with one cluster globally affect all classes regardless of r , while higher cluster number allow partial effects (e.g. $r = 0.5$), making two-cluster scenarios particularly manageable. However, further fragmentation in the stream (3–4 clusters) creates challenging data sparsity and variability conditions that lead to degradation in the performance of the approaches.

Figures 9 and 10 reveal an interesting finding: the majority of proactive adaptation mechanisms, implemented upon VFDT, demonstrate consistent accuracy improvement across all the examined configurations. The trends observed exhibit how an appropriately designed adaptation mechanism integrated with a base classifier, can enhance its efficacy when deal-

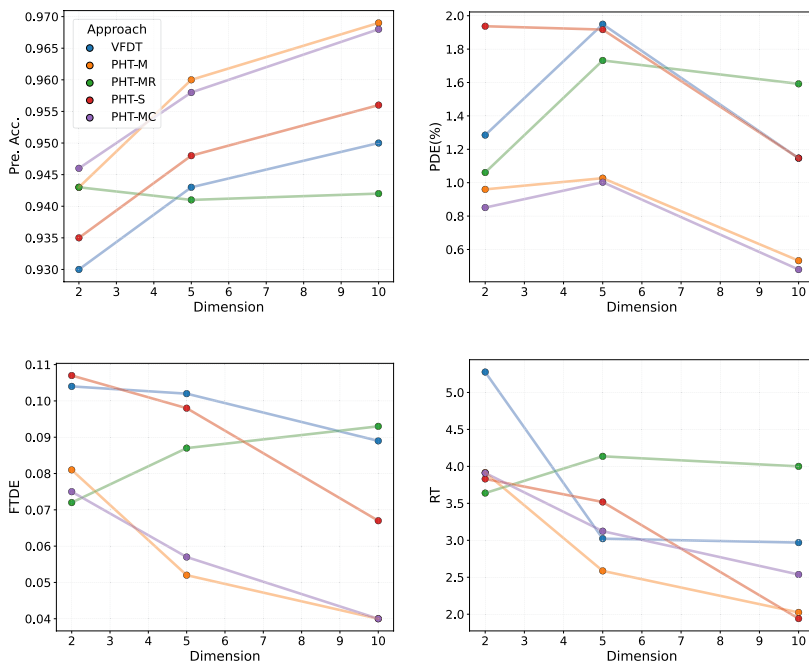


Fig. 9 Effect of dimensionality on different performance metrics across approaches

Table 5 Aggregated comparison of approaches across different numbers of clusters per class

Cluster/class	Model	Preq. Acc.	PDE (%)	FTDE	RT
1	VFDT	0.936 ± 0.060	1.401 ± 2.108	0.080 ± 0.129	3.256 ± 6.589
	PHT-M	0.957 ± 0.033	0.850 ± 1.385	0.035 ± 0.069	2.133 ± 3.707
	PHT-MR	0.960 ± 0.028	0.728 ± 1.151	0.028 ± 0.055	1.965 ± 3.836
	PHT-S	0.947 ± 0.046	1.458 ± 2.411	0.059 ± 0.102	1.741 ± 2.461
	PHT-MC	0.957 ± 0.032	0.803 ± 1.278	0.041 ± 0.074	2.604 ± 5.147
2	VFDT	0.957 ± 0.046	1.065 ± 1.790	0.081 ± 0.143	3.785 ± 8.101
	PHT-M	0.972 ± 0.029	0.533 ± 0.938	0.034 ± 0.069	1.986 ± 4.089
	PHT-MR	0.958 ± 0.037	1.258 ± 1.522	0.068 ± 0.096	3.026 ± 3.354
	PHT-S	0.964 ± 0.034	1.023 ± 1.597	0.063 ± 0.105	2.629 ± 4.713
	PHT-MC	0.972 ± 0.028	0.449 ± 0.769	0.037 ± 0.070	2.701 ± 5.397
3	VFDT	0.947 ± 0.054	1.486 ± 2.064	0.113 ± 0.157	4.111 ± 5.394
	PHT-M	0.962 ± 0.045	0.799 ± 1.249	0.069 ± 0.132	3.528 ± 6.580
	PHT-MR	0.942 ± 0.046	1.710 ± 1.645	0.107 ± 0.126	4.750 ± 4.406
	PHT-S	0.952 ± 0.047	1.818 ± 2.211	0.106 ± 0.138	3.692 ± 4.118
	PHT-MC	0.963 ± 0.041	0.683 ± 1.080	0.067 ± 0.125	3.589 ± 5.946
4	VFDT	0.923 ± 0.069	1.888 ± 2.209	0.119 ± 0.173	3.873 ± 5.145
	PHT-M	0.938 ± 0.059	1.178 ± 1.509	0.092 ± 0.161	3.722 ± 6.352
	PHT-MR	0.908 ± 0.059	2.150 ± 1.972	0.132 ± 0.153	5.958 ± 7.727
	PHT-S	0.924 ± 0.070	2.369 ± 3.248	0.135 ± 0.165	4.323 ± 4.866
	PHT-MC	0.937 ± 0.055	1.178 ± 1.466	0.086 ± 0.138	3.867 ± 6.905

The best value for each metric is highlighted in bold

ing with concept drift. This empirical evidence underscores once again the importance of adaptive strategies in helping base classifiers to maintain classification performance under evolving data streams.

Table 6 presents the results obtained across all scenario types for each approach. The *Chase* scenario represents the simplest but longest form of incremental drift studied, where clusters follow straight-line movements along predefined paths. In this scenario, PHT-MC achieves the highest performance in all metrics, closely followed by PHT-M (0.981 and 0.980 prequential accuracy, respectively).

Chase offers an ideal case study for proactive approaches since simple threshold adjustments are often sufficient to mitigate the impact of concept drift in this scenario. PHT-M and PHT-MC excel due to their robust threshold adaptation mechanisms, while PHT-MR struggles with its rotation mechanism, achieving the weakest results across all metrics. The invariant distance between clusters of points in this scenario makes the rotation heuristic of PHT-MR less reliable. Interestingly, PHT-S obtains competitive accuracy (0.976) despite lacking explicit threshold adjustment, which can be attributed to the fact that its synthetic samples generation becomes specially reliable when there are simple straight-line and non-overlapping movements.

The *Split* scenario fragments clusters into multiple portions, creating variable complexity depending on configuration. The outcomes observed in this setting are consistent with those previously reported. PHT-M and PHT-MC achieve the highest accuracy (0.971 and 0.970 respectively), while VFDT exhibits surprisingly competitive adaptation metrics. This apparent contradiction can be explained by the evaluation framework: degradations in PHT-M and PHT-MC are often too minor to be classified as degradation episodes (requiring > 5%

performance drops), while VFDT experiences more frequent measurable degradations, making the adaptation metrics potentially misleading if not interpreted carefully.

The difficulty of the *Split* scenario can vary substantially depending on the experimental configuration. In settings with high number of clusters, fragmenting them into smaller portions increases both the sparsity and variance of the data. High intra-class variance can directly impact the effectiveness of the proposed proactive adaptation mechanisms, as they typically rely on the extraction of information from the points closest to the decision boundaries. PHT-MR performs poorly (0.952 accuracy) due to its sensitivity to sparse scenarios. PHT-S also underperforms (0.965 accuracy) compared to the previous scenario. In contrast, PHT-M and PHT-MC maintain robust performance through their simple yet effective threshold adaptation mechanism, demonstrating their resilience regardless of the complexity faced.

Merge scenario gradually merges clusters belonging to the same class, conceptually similar to *Cross* scenario but without necessarily involving inter-class overlaps. In this setting, PHT-M achieves the best overall performance (0.968 prequential accuracy) and superior adaptation metrics (0.519% degradation episodes, 0.849 recovery time), outperforming PHT-MC particularly in recovery speed. PHT-MR, once again, exhibits the weakest results

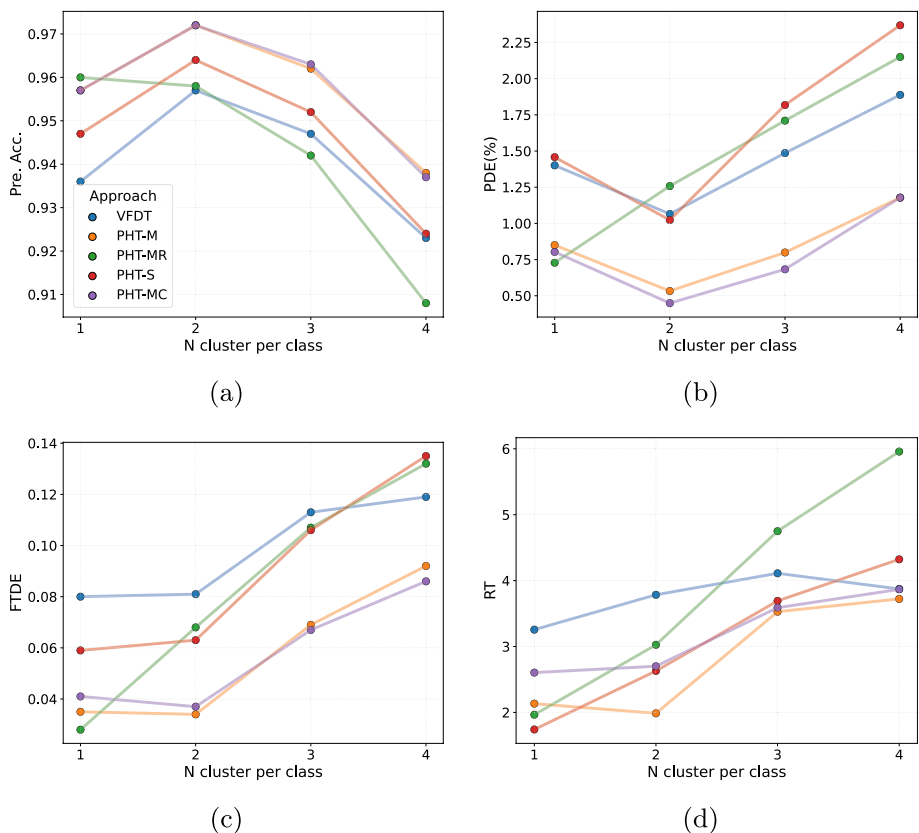


Fig. 10 Effect of number of clusters per class on different performance metrics across approaches

(0.949 prequential accuracy), reinforcing the instability that its rotation mechanism suffer in complex settings involving overlaps.

The scenario's difficulty varies significantly with configuration complexity, as some *Merge* settings do not sufficiently challenge the models, similar to *Split*. In more challenging configurations (> 2 clusters per class), performance differences become more pronounced, with the gap between VFDT and proactive approaches widening significantly, highlighting the importance of proactive adaptation mechanisms when dealing with complex configurations of Merge scenario.

Table 6 Aggregated comparison of approaches across different scenario types

Scenario	Model	Preq. Acc.	PDE (%)	FTDE	RT	
Chase	VFDT	0.972 $\pm$ 0.030	0.972 $\pm$ 1.430	0.044 $\pm$ 0.075	2.793 $\pm$ 4.094	
		PHT-M	0.980 $\pm$ 0.030	0.442 $\pm$ 1.313	0.014 $\pm$ 0.056	0.731 $\pm$ 1.723
	PHT-MR	0.966 $\pm$ 0.042	1.020 $\pm$ 1.380	0.050 $\pm$ 0.105	2.906 $\pm$ 3.574	
	PHT-S	0.976 $\pm$ 0.029	0.874 $\pm$ 1.612	0.034 $\pm$ 0.066	1.675 $\pm$ 2.463	
	PHT-MC	0.981 $\pm$ 0.029	0.391 $\pm$ 1.182	0.011 $\pm$ 0.049	0.509 $\pm$ 0.918	
	Split	VFDT	0.967 $\pm$ 0.021	0.182 $\pm$ 0.426	0.002 $\pm$ 0.004	0.167 $\pm$ 0.375
		PHT-M	0.971 $\pm$ 0.019	0.210 $\pm$ 0.447	0.002 $\pm$ 0.004	0.194 $\pm$ 0.399
		PHT-MR	0.952 $\pm$ 0.031	1.024 $\pm$ 1.733	0.044 $\pm$ 0.082	1.951 $\pm$ 3.181
		PHT-S	0.965 $\pm$ 0.024	0.519 $\pm$ 0.777	0.019 $\pm$ 0.040	1.399 $\pm$ 2.784
		PHT-MC	0.970 $\pm$ 0.019	0.182 $\pm$ 0.426	0.002 $\pm$ 0.004	0.167 $\pm$ 0.375
Merge	VFDT	0.965 $\pm$ 0.031	0.687 $\pm$ 1.197	0.026 $\pm$ 0.063	1.191 $\pm$ 2.383	
	PHT-M	0.968 $\pm$ 0.026	0.519 $\pm$ 0.913	0.018 $\pm$ 0.060	0.849 $\pm$ 1.891	
	PHT-MR	0.949 $\pm$ 0.036	1.108 $\pm$ 1.201	0.052 $\pm$ 0.086	2.784 $\pm$ 4.669	
	PHT-S	0.963 $\pm$ 0.034	0.898 $\pm$ 1.404	0.044 $\pm$ 0.094	1.731 $\pm$ 2.749	
	PHT-MC	0.966 $\pm$ 0.027	0.519 $\pm$ 0.928	0.024 $\pm$ 0.066	1.390 $\pm$ 3.220	
Cross	VFDT	0.859 $\pm$ 0.051	3.998 $\pm$ 2.107	0.321 $\pm$ 0.124	10.873 $\pm$ 8.390	
	PHT-M	0.910 $\pm$ 0.054	2.189 $\pm$ 1.257	0.196 $\pm$ 0.149	9.595 $\pm$ 6.930	
	PHT-MR	0.901 $\pm$ 0.055	2.694 $\pm$ 1.721	0.190 $\pm$ 0.132	8.058 $\pm$ 6.808	
	PHT-S	0.882 $\pm$ 0.055	4.377 $\pm$ 3.098	0.266 $\pm$ 0.124	7.580 $\pm$ 4.925	
	PHT-MC	0.913 $\pm$ 0.049	2.020 $\pm$ 1.112	0.193 $\pm$ 0.122	10.695 $\pm$ 7.180	

The best value for each metric is highlighted in bold

The *Cross* scenario introduces an interesting challenge to the approaches, as the incremental concept drift inevitably leads to overlap between two clusters of different classes, regardless of the experimental configuration. These overlap situations are challenging for proactive approaches since, even if the drift can be anticipated, correctly classifying all instances remains nearly impossible. These critical situations can also affect the effectiveness and stability of the proactive adaptation mechanism. In this setting, the recovery time substantially increases compared to other scenarios, highlighting the complexity and impact of the overlap phenomenon.

The performance gap between reactive and proactive approaches is particularly pronounced in this scenario. PHT-MC achieves the highest prequential accuracy (0.913) and best PDE (2.020%), nearly half of VFDT's degradations (3.998%). Remarkably, all proactive approaches outperform VFDT's recovery time, with PHT-S showing the fastest recovery (7.580 vs VFDT's 10.873).

PHT-S's fast recovery can be attributed to its synthetic data generation mechanism, which trains shadow trees using displaced synthetic points. When the overlap occurs, these synthetic points could simulate post-overlap conditions, enabling faster adaptation during the critical moment. However, its high degradation episodes (4.377%) suggest frequent but quickly resolved performance drops.

However, recovery time can be misleading if interpreted without considering degradation episode frequency. VFDT experiences significantly more degradation episodes (3.998%) than proactive approaches (PHT-M: 2.189%, PHT-MC: 2.020%) due to the *Cross* scenario's nature, where clusters exchange positions through overlapping midpoints. As illustrated in Fig. 11, VFDT begins experiencing short degradation episodes even before overlap occurs and continues suffering short-term episodes after, while PHT-M suffer fewer degradation episodes, but they tend to last longer (its degradation episodes represent the overlap critical moment). This behavior resembles high sensitive drift detectors that trigger frequent false alarms, while appearing effective based on posterior metrics, they often overreact to noise reducing their practical reliability. Similarly, VFDT, recovery time may reflect numerous short degradations rather than effective adaptation to the overlap situation.

Figure 12 provides a detailed analysis of the *Cross* scenario using sliding windows of 500 instances. The stabilization values for each approach correspond to the point (averaged from window indexes 20 and 50 on each configuration) at which each approach begins to experience a degradation episode due to the ongoing overlap situation. Approximately, at window index 50, each approach starts its recovery phase, as the overlapping cluster start to separate. The results show that proactive approaches recover earlier (around window index 70) than VFDT (window index 80), demonstrating superior adaptation during the critical overlap period, and faster recovery after the critical phase.

Our results show that the proactive strategies PHT-M and PHT-MC presented robust performance regardless the dimensionality, number of cluster or type of scenario studied. These advantages are evident in both, prequential accuracy and adaptation-related metrics, specially when compared against the reactive baseline. Notably, VFDT suffered from slower recoveries after accuracy drops, significantly more degradation episodes, and lower performance than the rest of the proactive approaches on average. These findings provide strong evidence supporting the effectiveness of proactive adaptation mechanisms on improving VFDT results, regardless the conditions, in incremental concept drifts, and therefore answering **RQ2**.

6.2 Overall Comparison

This subsection presents and discusses the overall performance of each proactive approach, along with the reactive baseline, VFDT. Additionally, we examine how varying incremental drift speeds affect performance and conduct hyperparameter analysis of the different approaches. The goal is to provide a comprehensive overview of their behavior across all scenarios and experimental settings.

Table 7 summarizes the aggregated results for the prequential accuracy and adaptation-related metrics studied in our experimentation across all streams. The best value for each metric is highlighted in bold. In terms of prequential accuracy, PHT-MC achieved the best overall performance (0.9574), showing a notable improvement over the reactive baseline, VFDT (0.9406). As expected, PHT-M obtained similar results (0.9572), positioning itself as the second-best approach in this metric. The lowest prequential accuracy was recorded by the VFDT (0.9406), although the difference compared to PHT-MR and PHT-S is relatively small and not statistically significant. These observations support the findings discussed in the previous sections regarding the instability introduced by certain proactive adaptation strategies.

Table 7 further reinforces the effectiveness of PHT-M and PHT-MC. Both approaches experience the lowest percentage of degradation episodes (0.840 and 0.778 respectively) and fastest recovery times, with PHT-M showing slight superiority in recovery speed. The reactive baseline presents weak performance across all adaptation-related metrics, closely

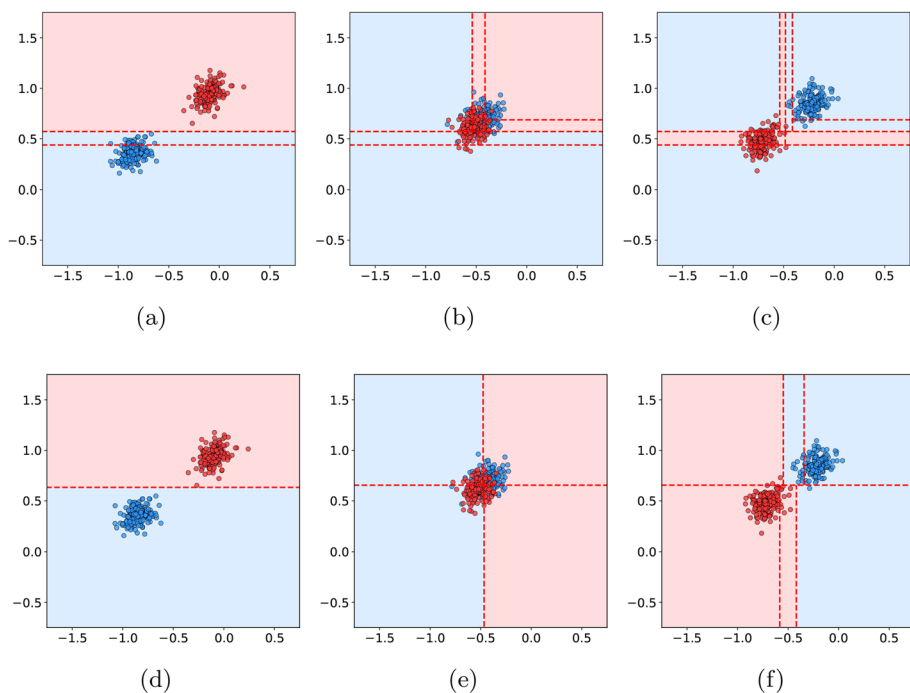
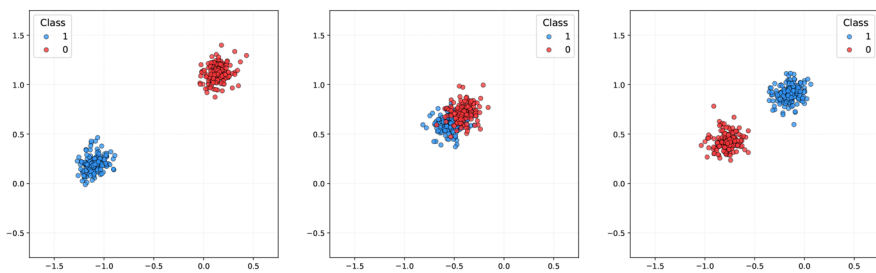


Fig. 11 Representation of two dimensional *Cross* scenario with one cluster per class. **a–c** represent VFDT's adaptation to the critical situation of the scenario, when the overlap occur. **d–f** represent PHT-M's adaptation to the overlap situation

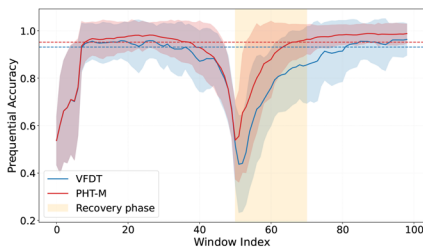
followed by PHT-S and PHT-MR. The latter obtains the worst recovery time (3.925), what is consistent with its previously discussed instable behavior. These findings support the strength of PHT-M and PHT-MC proactive adaptation mechanism, which remains simple yet robust across all experimental settings.

As stated in Sect. 5.2, we conducted multiple statistical tests to assess the reliability of our results. The outcomes of these tests are summarized in Fig. 13. It is worth to highlight that the tests provide statistical evidence of significant differences between the reactive baseline, VFDT, and the two top-performing approaches, PHT-M and PHT-MC. In contrast, the performance differences between the VFDT and the weaker proactive strategies, PHT-MR and PHT-S, were not found to be statistically significant.

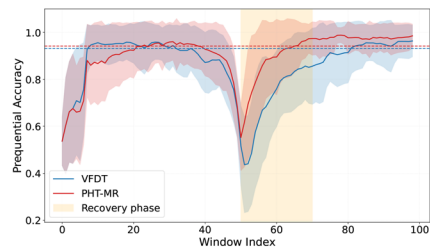
These findings provide an overall perspective of the behavior of the approaches in the incremental concept drift scenarios studied, allowing us to address **RQ1**: PHT-M and PHT-



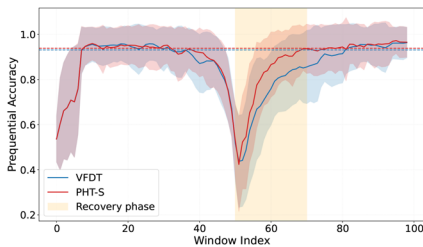
(a) Instance of *Cross* scenario representing different periods of the overlap situation. Samples corresponding to window index 10, 45 and 70 respectively.



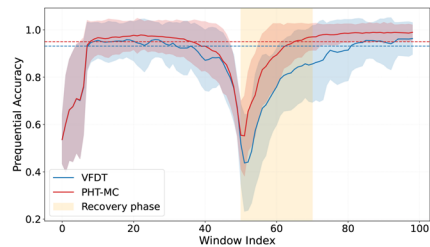
(b) VFDT vs PHT-M



(c) VFDT vs PHT-MR



(d) VFDT vs PHT-S



(e) VFDT vs PHT-MC

Fig. 12 Mean prequential accuracy per 500 samples between VFDT and each proactive approach across all *Cross* scenario configurations. **a** presents the data streams across the cross concept drift and **b-d** illustrates VFDT and PHT variants performance across the stream. Shaded area indicates $\pm$ standard deviation. Dashed horizontal lines represent the stabilization values

MC demonstrate evident superiority over all other approaches, highlighting the effectiveness of a simple mechanism that continuously maximizes the class separability. While the reactive baseline was competitive in terms of prequential accuracy in some specific settings, it exhibited the weakest results in all the metrics in the overall comparison, remarking its inability to anticipate simple incremental concept drifts. It suffers almost twice as many degradation episodes (1.460) compared to PHT-M and PHT-MC. Notably, the recovery time presented by PHT-M and PHT-MC is lower than VFDT's, meaning that the number of instances that the reactive baseline takes on average to recover from a significant drop in accuracy is higher than those required by the mentioned proactive approaches. This further remarks on the effectiveness of proactive approaches when dealing with incremental concept drifts and mitigating the negative effects they can cause.

6.2.1 Drift Speed Analysis

Table 8 presents the prequential accuracy results across the different datasets accelerated through α parameter. These streams were created by making the original concept drift last lower. The results clearly reveal how all approaches degraded performance as drift speed increases (α decreases), with the most challenging condition being $\alpha = 0.25$. Under these conditions, concept drifts complete in significantly fewer samples, providing limited time for proactive adaptation mechanisms to anticipate effectively.

PHT-M and PHT-MC maintain superior performance across all speed conditions, demonstrating remarkable robustness regardless of the incremental concept drift speed. Notably, the performance gap between the top proactive approaches and the reactive baseline maintains stable regardless the speed. This experimentation has been executed with a proactive adaptation window of 1000 samples ($|W| = 1000$), as default. However, it is reasonable to expect that faster incremental concept drifts could benefit from more frequent proactive adaptations, as the default window size may be suboptimal for rapidly changing conditions. A detailed discussion of this hyperparameter optimization is provided in Sect. 6.2.2.

Table 9 presents the overall results obtained in our experimentation over *abrupt concept drifts*. This type of concept drift, as mentioned previously, imposes important limitations on our proactive approaches as there is insufficient trajectory information during the drift occurrence to anticipate or predict the distribution changes to mitigate the negative consequences. Naturally, abrupt concept drifts should be addressed by other families of proactive approaches (e.g. history-based). Under these conditions, we expected the traditional reactive VFDT to obtain results similar to, if not better than, the proactive ones.

Table 7 Aggregation comparison of prequential accuracy and adaptation-related metrics between proactive approaches and VFDT

The best value for each metric is highlighted in bold

Model	Preq. Acc.	PDE (%)	FTDE	RT
VFDT	0.9406 $\pm$ 0.059	1.460 $\pm$ 2.059	0.098 $\pm$ 0.152	3.756 $\pm$ 6.389
PHT-M	0.9572 $\pm$ 0.044	0.840 $\pm$ 1.302	0.058 $\pm$ 0.117	2.842 $\pm$ 5.371
PHT-MR	0.9420 $\pm$ 0.048	1.461 $\pm$ 1.677	0.084 $\pm$ 0.120	3.925 $\pm$ 5.326
PHT-S	0.9466 $\pm$ 0.053	1.667 $\pm$ 2.477	0.091 $\pm$ 0.133	3.096 $\pm$ 4.246
PHT-MC	0.9574 $\pm$ 0.042	0.778 $\pm$ 1.200	0.057 $\pm$ 0.108	3.190 $\pm$ 5.883

These results partially confirm our expectations: All approaches converge to similar accuracy levels (≈ 0.82) indicating that adaptation relies mainly on reactive components under abrupt conditions. However, most proactive approaches still achieve marginally better results than VFDT, with PHT-S obtaining the highest accuracy (0.8271). This suggests that even under abrupt conditions, proactive mechanisms retain some utility: threshold optimization in PHT-M and PHT-MC provides slight benefits by maintaining better decision boundary positioning, while PHT-S's synthetic data generation enables faster retraining during critical transitions, reducing time under performance degradation.

6.2.2 Hyperparameter Sensitivity Analysis

As mentioned in Sect. 6.2.1, when dealing with faster incremental concept drifts, the frequency of proactive adaptation becomes crucial for anticipating or mitigating the consequences of distribution changes. The frequency of the proactive adaptation is managed by the proactive window size, $|W| = 1000$ by default. To investigate the relationship, we conducted a sensitivity analysis of the proactive window size parameter, testing values $|W| \in \{500, 1000, 2500, 5000\}$, on our original dataset, which is characterized by relatively slow incremental concept drift.

Table 10 reveals interesting patterns regarding proactive adaptation frequency. While there is a general tendency toward better performance with more frequent adaptations (smaller $|W|$), the optimal window size varies across approaches, demonstrating that each

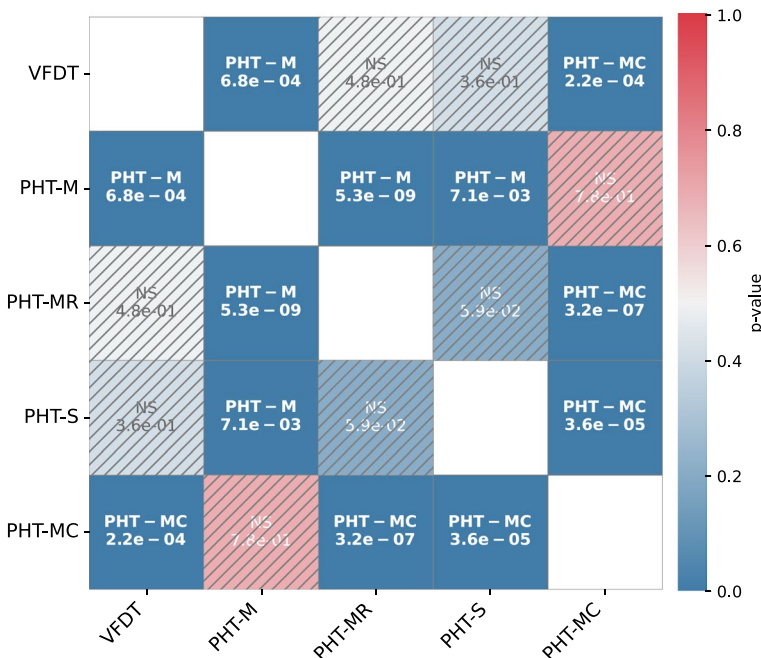


Fig. 13 Heatmap showing the results of pairwise statistics tests comparing the prequential accuracy of each pair of approaches, using a significant level of $\alpha = 0.05$. Each cell displays the statistically superior approach (when significant), along with the corresponding p-value. The no significant statistics tests are illustrated with hatch background

Table 8 Prequential accuracy comparison across different drift speeds (α values)

Model	$\alpha = 0.25$	$\alpha = 0.50$	$\alpha = 0.75$	$\alpha = 1.0^*$
VFDT	0.8475	0.9017	0.9256	0.9407
PHT-M	0.8615	0.9247	0.9468	0.9572
PHT-MR	0.8287	0.8980	0.9262	0.9420
PHT-S	0.8472	0.9058	0.9334	0.9466
PHT-MC	0.8634	0.9229	0.9456	0.9574

The best value for each is highlighted in bold.

The lower the α is, the faster the concept drifts presented in the dataset are $\alpha = 1.0$. *The original dataset is generated with

Table 9 Aggregation comparison of prequential accuracy and adaptation-related metrics between proactive approaches and VFDT under abrupt concept drifts

Model	Preq. Acc.	PDE (%)	FTDE	RT
VFDT	0.8239 $\pm$ 0.107	10.526 $\pm$ 4.962	0.200 $\pm$ 0.124	1.925 $\pm$ 1.054
PHT-M	0.8249 $\pm$ 0.098	8.947 $\pm$ 4.993	0.205 $\pm$ 0.115	2.375 $\pm$ 1.101
PHT-MR	0.8104 $\pm$ 0.116	10.000 $\pm$ 5.792	0.216 $\pm$ 0.137	2.175 $\pm$ 1.003
PHT-S	0.8271 $\pm$ 0.097	8.947 $\pm$ 5.576	0.189 $\pm$ 0.141	2.333 $\pm$ 2.121
PHT-MC	0.8252 $\pm$ 0.098	9.474 $\pm$ 5.436	0.216 $\pm$ 0.122	2.375 $\pm$ 1.101

The best value for each metric is highlighted in bold

mechanism has its own sensitivity profile. The performance differences across window sizes are relatively modest (typically $< 1\%$), indicating that while frequency matters, the approaches demonstrate reasonable robustness to this hyperparameter choice. This is consistent with our expectations, as trajectory-based proactive approaches benefit from frequent adjustments but may suffer from instability if window information is insufficient for reliable estimations. However, the choice of this hyperparameter would depend on the characteristics of the stream. Under accelerated drift conditions, proactive approaches would likely benefit from smaller window sizes to maintain their anticipatory advantages.

Some of the proactive approaches presented use hyperparameters that can significantly impact their performance depending on stream configuration, as observed in previous sections. Specifically, PHT-S demonstrates variable behavior, sometimes obtaining very poor results while occasionally achieving exceptionally good performance. This inconsistency motivated us to conduct a brief analysis of its main hyperparameter: the threshold τ that determines when to trigger the training of a shadow tree. A higher threshold value makes the mechanism more sensitive, triggering shadow tree creation earlier during potential drift situations.

Table 11 presents a sensitivity analysis of PHT-S's threshold parameter τ on our original dataset. The results reveal that PHT-S performance slightly improves with a higher threshold value (more sensitive triggering), achieving optimal prequential accuracy (0.9473) with threshold 0.75. However, the performance difference across threshold values are relatively modest, indicating that PHT-S's proactive adaptation mechanism remains reasonably robust within the tested range. While the improvements are marginal for our experimental framework, PHT-S could benefit from further hyperparameter tuning to optimize its performance

for specific stream configurations, with adaptive hyperparameter approaches representing a promising future work direction for improving proactive approaches.

Similarly, we conducted a sensitivity analysis for PHT-MC's threshold parameter τ , which controls when the subtree creation mechanism is triggered. Following the same principle as PHT-S, higher τ values make the approach more sensitive to potential drift signals. Table 12 shows the performance across different threshold values in our original dataset. The results indicate that lower threshold values (less sensitive triggering) yield better performance, with $\tau = 0.001$ achieving the highest prequential accuracy (0.9579). This suggests that conservative subtree creation may be preferable to avoid premature adaptations, even though performance differences are minimal (less than 0.003 in prequential accuracy across all tested thresholds).

These results reinforce the robustness of the proposed proactive approaches. Despite being sensitive to different stream configurations, they demonstrate stability when dealing with different hyperparameter values, as the performance differences remain statistically insignificant. However, as observed in both PHT-S and PHT-MC analyses, the optimal approach would be to develop adaptive hyperparameter mechanisms that dynamically adjust these thresholds based on stream characteristics. Such adaptive strategies could potentially enhance performance by automatically tuning parameters according to drift speed, complexity, and other stream properties, representing a promising direction for future research.

6.2.3 Resource Analysis

Further analysis was conducted to evaluate the impact of the proposed proactive adaptation mechanisms on memory consumption and execution time over the original dataset. It is important to note that while all experiments were conducted on the same machine under controlled conditions, minor timing and memory variations may occur due to external factors such as system load and background processes. Therefore, while these results are coherent and have been carefully validated, they should be considered indicative rather than absolute measurements. Table 13 presents the average memory usage and execution time for each of the approaches.

We observe that PHT-M achieved the lowest memory consumption, outperforming all other approaches by a substantial margin, with PHT-S ranking second. Note that the standard Hoeffding tree (VFDT) exhibited the highest memory consumption, closely followed by PHT-MC.

This high sources consumption by VFDT is consistent with the previously observed behavior and its nature. When dealing with continuous movement of the clusters, VFDT must frequently create new split nodes in order to adapt, which results in elevated memory usage. In contrast, proactive approaches offer an advantage in this regard, as their mecha-

Table 10 Prequential accuracy across different proactive window sizes on the original dataset. * $|W| = 1000$ is the value by default in our experimentation

The best value for each size is highlighted in bold

Model	$ W = 500$	* $ W = 1000$	$ W = 2500$	$ W = 5000$
VFDT	0.9407	0.9407	0.9407	0.9407
PHT-M	0.9582	0.9572	0.9549	0.9510
PHT-MR	0.9397	0.9420	0.9431	0.9437
PHT-S	0.9464	0.9466	0.9452	0.9435
PHT-MC	0.9569	0.9574	0.9546	0.9504

Table 11 PHT-S performance across different threshold values over the original dataset

Thresh- old (τ)	Preq. Acc.	PDE (%)	FTDE	RT
0.10	0.9454 $\pm$ 0.052	1.780 $\pm$ 2.306	0.094 $\pm$ 0.129	3.047 $\pm$ 3.635
0.25	0.9464 $\pm$ 0.051	1.706 $\pm$ 2.249	0.095 $\pm$ 0.140	3.322 $\pm$ 4.667
*0.50	0.9466 $\pm$ 0.053	1.671 $\pm$ 2.476	0.091 $\pm$ 0.133	3.098 $\pm$ 4.245
0.75	0.9473 $\pm$ 0.052	1.632 $\pm$ 2.471	0.086 $\pm$ 0.129	2.976 $\pm$ 4.280
1	0.9443 $\pm$ 0.057	1.790 $\pm$ 3.052	0.083 $\pm$ 0.124	2.688 $\pm$ 3.565

The best value for each metric is highlighted in bold

*By default, the threshold is fixed to 0.5

Table 12 PHT-MC performance across different threshold values (τ) over the original dataset

Threshold (τ)	Preq. Acc.	PDE (%)	FTDE	RT
0.001	0.9579 $\pm$ 0.041	0.784 $\pm$ 1.200	0.057 $\pm$ 0.108	3.215 $\pm$ 6.355
*0.008	0.9574 $\pm$ 0.042	0.775 $\pm$ 1.195	0.058 $\pm$ 0.109	3.236 $\pm$ 6.052
0.015	0.9561 $\pm$ 0.043	0.807 $\pm$ 1.259	0.060 $\pm$ 0.114	3.227 $\pm$ 5.848
0.030	0.9555 $\pm$ 0.044	0.882 $\pm$ 1.321	0.064 $\pm$ 0.118	3.091 $\pm$ 5.474

The best value for each metric is highlighted in bold

*By default, the threshold is fixed to 0.008

nism to adjust decision boundaries, allow them to mitigate the memory usage while adapting effectively.

In terms of execution time, the efficiency of the VFDT is evident, requiring significantly less processing time than the proactive approaches (7.28 s). This result aligns with our expectations, given the additional computational overhead introduced by the proactive adaptation mechanism. Nevertheless, PHT-M achieved the second-best execution time (10.04 s), what once again highlight its powerful ability to maintain a balance between resource usage and predictive accuracy. PHT-MR shines because of its inefficiency mechanism, which required around 2.5 times more execution time on average than the reactive baseline. The additional overhead introduced by the PHT-MC mechanism is reflected in its high execution time (16.73 s), making it the second most time-consuming approach in the study.

The results provided in Table 13 offer a clear overview of the efficiency aspect for all the approaches in our experimentation, addressing **RQ3**. As expected, VFDT stands as the faster approaches from all the compared, as it does not count with additional overhead presented by extra adaptation mechanisms. PHT-M presents almost double execution time than the presented by the reactive baseline, but significantly outperforms the rest of the proactive approaches. Once again, this is coherent with the nature of PHT-M adaptation mechanism, which is especially simple in comparison with the rest of proactive strategies studied.

Oppositely, the memory management is less in favor of the reactive baseline, which required the highest amount of memory by average to adapt to the studied concept drifts (239.92 MB on average). The most memory-efficiency approach is PHT-M (166.05 MB), which aligns with its simple mechanism that does not need special amount of data, and its ability to enhance split node management.

6.3 Real-World Dataset Overall Analysis

An additional study using real-world datasets helps us understand the suitability of the proposed proactive approaches under uncertain drift conditions that may not follow linear assumptions. These experiments serve as a complex challenge for our approaches, whose proactive adaptation mechanisms are designed for incremental concept drifts with trajectories approximable by linear transformations of the decision boundaries. In this context, we expect our proactive approaches to struggle to maintain their advantage against VFDT when faced with irregular, non-linear and chaotic drift patterns characteristic of real-world scenarios. Due to the nature of the streams selected, we have fixed $|W| = 500$ to allow more frequent proactive adjustments as most of the considered real-world streams were particularly short.

Table 14 presents the aggregated results across all real-world datasets, revealing a different performance landscape compared to our synthetic experiments. The performance advantages of proactive approaches are significantly diminished in real-world scenarios, aligning with our expectations. However, the performance gap between the different approaches is not significant, obtaining all the approaches similar results. Interestingly, PHT-S obtains the highest results in all the metrics considered. This can be explained by its synthetic data generation methodology which allows the decision tree to leverage training on additional samples that approximate real data stream trends. The performance drop of PHT-M and PHT-MC, which dominated synthetic scenarios, is expected as these algorithms rely heavily on the linear nature of the distribution changes. The uncertain nature of real-world streams makes these approaches underperform in comparison to previous results.

These findings highlight an important limitation of current trajectory-based proactive approaches and underscore the need for more proactive adaptive mechanisms that can handle non-linear drift patterns. The results address **RQ4**, demonstrating that while proactive approaches maintain competitive performance in real-world scenarios, their synthetic advantages do not fully translate to practical applications, emphasizing the importance of developing more robust proactive strategies for real-world deployment.

6.4 Generalization of Proactive Approaches in Other Tree-Based Classifiers

The experimental analysis conducted throughout this study has demonstrated the effectiveness of proactive adaptation strategies to enhance the performance of a basic decision tree such as the Hoeffding Tree (HT) in the presence of gradual controlled concept drifts. Although, HT remains a foundational model in the field of data streams, it has been significantly improved in the literature through sophisticated variants (Hulten et al., 2001; Gama et al., 2003; Bifet and Gavalda, 2009).

Table 13 Aggregation comparison of resources-related metrics between proactive approaches and VFDT

The best value for each metric is highlighted in bold

Model	Memory (MB)	Execution Time (s)
VFDT	239.92 ± 233.91	7.28 ± 3.38
PHT-M	166.05 ± 171.73	10.04 ± 7.12
PHT-MR	202.28 ± 200.44	17.73 ± 16.01
PHT-S	192.63 ± 156.29	15.76 ± 11.85
PHT-MC	227.61 ± 281.12	16.73 ± 20.57

To evaluate the flexibility and generalizability of the proposed proactive strategies, we extended our investigation to more advanced tree-based classifiers. Specifically, we selected two widely recognized models: Hoeffding Adaptive Tree (HAT) (Bifet and Gavalda, 2009) and Extremely Fast Decision Tree (EFDT) (Manapragada et al., 2018).

Given the architectural complexity and internal mechanisms of these models, we opted to incorporate only the least intrusive and most effective proactive adaptation mechanism, threshold adaptation mechanism (the used in PHT-M). This choice ensures minimal conflict with the models' internal processes while allowing the evaluation of incorporating proactive mechanisms into these models, resulting in Proactive HAT (PHAT-M) and Proactive EFDT (PEFDT-M).

Table 15 summarizes the results obtained by the two standard models and their proactive version. These values are computed as the aggregation of results across all data streams and experimental settings considered in this work (without including the dataset used in additional experimentation), ensuring a consistent comparison with the analysis presented in Sect. 6.2. PHAT-M obtained statistically significant better results in terms of prequential accuracy than HAT (0.9689 vs 0.9627), and beats its reactive version in all adaptation-related metrics. EFDT and its proactive version, PEFDT-M, do not show a statistically significant difference in terms of prequential accuracy; however, in our scenarios, the proactive version is able to slightly outperform the reactive model in all the metrics with the exception of recovery time, where EFDT showed minimal better results. In Table 16, the results obtained from the resource-related study are provided.

With this study we aim to prove the convenience and benefit of integrating proactive adaptation mechanisms in current state-of-the-art models. Interestingly, we can see that HAT itself obtained better results than most of the proactive approaches based on the HT. However, this comparison should be interpreted with caution, as the primary objective of this work is not to identify the single best-performing model, but rather to show how proactive strategies can enhance the adaptability of existing classifiers to concept drift. With this final generalization experiment, we were able to prove that proactive mechanisms such as the proposed could lead to a significant improvement to state-of-the-art tree-based models, therefore addressing RQ5. Finally, Fig. 14 presents a stacked radial comparison of all classifiers and approaches evaluated, considering all selected metrics.

Table 14 Aggregation comparison of prequential accuracy and adaptation-related metrics between proactive approaches and VFDT in real-world streams

Proactive window size fixed to $|W| = 500$
The best value for each metric is highlighted in bold

Model	Preq. Acc.	PDE (%)	FTDE	RT
VFDT	0.7654 ± 0.191	15.824 ± 26.205	0.074 ± 0.142	0.424 ± 0.810
PHT-M	0.7656 ± 0.191	13.949 ± 17.992	0.079 ± 0.155	0.475 ± 0.930
PHT-MR	0.7456 ± 0.199	15.379 ± 18.744	0.075 ± 0.145	0.592 ± 1.227
PHT-S	0.7661 ± 0.191	11.726 ± 15.085	0.070 ± 0.131	0.379 ± 0.714
PHT-MC	0.7646 ± 0.191	17.086 ± 23.558	0.074 ± 0.142	0.424 ± 0.810

Table 15 Aggregation comparison of prequential accuracy and adaptation-related metrics between HAT, EFDT and their proactive version

Model	Preq. Acc.	PDE (%)	FTDE	RT
HAT	0.9627 ± 0.035	1.193 ± 1.817	0.057 ± 0.096	2.379 ± 3.993
PHAT-M	0.9689 ± 0.031	0.693 ± 1.254	0.040 ± 0.081	2.110 ± 4.068
EFDT	0.9350 ± 0.096	1.851 ± 2.488	0.164 ± 0.222	8.821 ± 22.982
PEFDT-M	0.9358 ± 0.095	1.830 ± 2.406	0.163 ± 0.220	8.837 ± 22.996

The best value for each metric is highlighted in bold

Table 16 Aggregation comparison of resources-related metrics between HAT, EFDT and their proactive version

Model	Memory (MB)	Execution Time (s)
HAT	183.93 ± 126.64	11.39 ± 7.31
PHAT-M	151.11 ± 111.67	13.56 ± 9.10
EFDT	196.10 ± 146.40	10.29 ± 6.67
PEFDT-M	197.67 ± 149.33	13.29 ± 9.05

The best value for each metric is highlighted in bold

7 Limitations and Future Directions

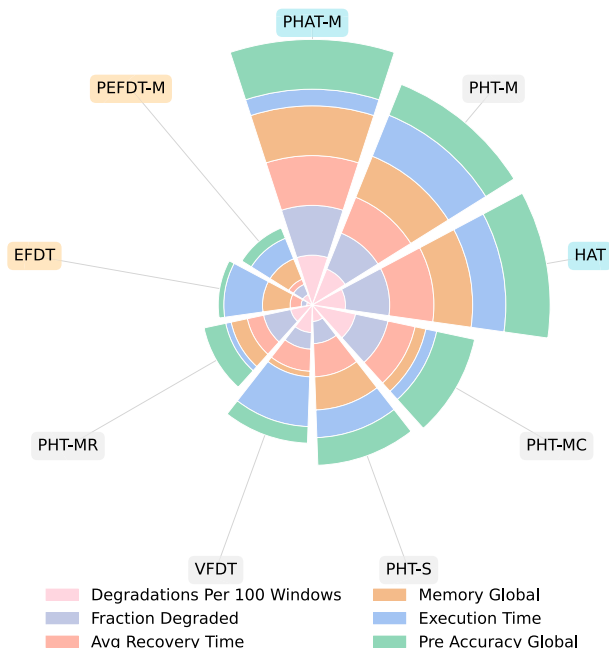
While our comprehensive evaluation demonstrates the effectiveness of proactive adaptation strategies, several important limitations must be acknowledged. Most notably, proactive approaches rely fundamentally on the assumption that concept drift can be anticipated to some degree based on historical patterns or trends, limiting their applicability to scenarios where drift follows predictable trajectories. As such, they are generally ineffective in the presence of abrupt or specially complex distribution changes, where reactive mechanisms must assume full responsibility for adaptation.

We have focused our experimentation on incremental concept drifts, as the sudden nature of abrupt concept drifts provides minimal opportunities for anticipation using the trajectory-based estimation methods presented, as the results shown. However, we acknowledge that some proactive approaches not explored in this work, such as those based in meta-learning, transfer learning, or even hybrid proactive-reactive methods (e.g. DetectA), may offer viable solutions.

Another significant challenge emerges in high-speed data streams, where concept drift evolves rapidly and instances arrive at a high rate. In such environments, fixed-size proactive adaptation windows may become obsolete quickly, requiring more frequent updates using smaller windows that reduce the information available for reliable trajectory estimation. Our evaluation also revealed that certain proactive strategies may perform exceptionally well under specific conditions but fail entirely in others. While methods like PHT-S or PHT-MC showed some dependence on hyperparameters, they produced robust results without significant differences, even though these parameter-sensitive approaches could benefit from self-tuning strategies.

Based on our evaluation, we strongly recommend the application of proactive adaptation strategies in scenarios characterized by gradual and minimally predictable concept drifts, where they consistently outperformed reactive baselines. This recommendation is further supported by our real-world validation experiments, which demonstrate that while proactive approaches maintain competitive performance under uncertain drift conditions, their

Fig. 14 Stacked radial ranking of all the reactive classifiers and their proactive versions. All metrics contribute equally to the final ranking



synthetic advantages are significantly diminished when faced with non-linear and irregular drift patterns. This outcome aligns with the fundamental design philosophy of our proposed methods: rather than attempting to solve all concept drift scenarios, we present targeted proactive adaptation mechanisms specifically optimized for incremental drifts with predictable trajectories.

Future research should prioritize the development of adaptive parameter self-tuning mechanisms to improve reliability across different stream configurations. We particularly encourage exploration of hybrid models that combine proactive strategies with state-of-the-art reactive methods, exploiting proactivity when change anticipation is feasible and falling back to reactivity when sudden drifts make anticipation unfeasible. Additionally, investigating the integration of meta-learning and transfer learning approaches could extend proactive capabilities to abrupt drift scenarios, while adaptive window sizing mechanisms could address challenges posed by high-speed data streams in proactive adaptation mechanism. Such dual adaptability between proactive and reactive methods could represent a meaningful advancement towards more robust and versatile solutions for learning in evolving data streams.

8 Conclusion

In this work, we introduced a novel categorization of adaptation mechanisms for handling concept drift, distinguishing between proactive and reactive strategies in non-stationary data streams. While these terms have been occasionally used to describe models, to the best of our knowledge, they had not been rigorously defined or systematically analyzed before.

We presented and evaluated several proactive adaptation mechanisms built upon the VFDT classifier, providing a comprehensive overview of their potential. Our study primarily addresses incremental concept drifts, as this type of drift allows for the possibility of anticipation, although we extended our experimentation to include varying incremental drift speeds and abrupt concept drifts scenarios. We evaluated these approaches across diverse synthetic data streams and real-world datasets, providing valuable insights into how proactive mechanisms can effectively address challenges that impact well-established baselines like VFDT.

Our findings demonstrate the importance of proactivity in mitigating the negative impact of concept drift on classification performance. These findings also highlight the absence of proactive strategies in the literature. At the same time, we acknowledge several limitations of proactive strategies: they rely on assumptions or estimations about data stream trends, which may not be always accurate, and they may exhibit instability under certain conditions. Therefore, we argue that proactive adaptation deserves further attention but should be carefully analyzed to properly address the concept drift on certain data stream settings.

As future work, we plan to explore the integration of proactive strategies into state-of-the-art classifiers and ensembles methods, which have shown superior performance in the data stream learning. Additionally, we aim to design a novel classifier with an implicit proactive adaptation mechanism, capable of handling real-world concept drifts by anticipating changes in the data distribution, building on the promising capabilities shown by the approaches studied in this work.

Author Contributions J.V.G.C. designed the study, implemented the proposed methods, conducted the experiments, and wrote the main manuscript text. G.J.A. closely supervised and contributed the writing and experimental process. A.C. provided overall supervision, guided the research direction, and contributed with revisions to the manuscript. All authors reviewed and approved the final version of the manuscript.

Funding No funding was received for conducting this study.

Data Availability No datasets were generated or analysed during the current study.

Code availability All the code necessary to reproduce the experimentation described in this study is publicly available on Github (<https://github.com/Vgueda/ProactiveHT>).

Declarations

Conflict of interest The authors have no relevant financial or non-financial interests to disclose.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Agrahari, S., & Singh, A. K. (2022). Concept drift detection in data stream mining: A literature review. *Journal of King Saud University - Computer and Information Sciences*, 34(10), 9523–9540.
- Aguiar, G. J., & Cano, A. (2024). A comprehensive analysis of concept drift locality in data streams. *Knowledge-Based Systems*, 289, Article 111535.
- Aguiar, G., Krawczyk, B., & Cano, A. (2024). A survey on learning from imbalanced data streams: Taxonomy, challenges, empirical study, and reproducible experimental framework. *Machine Learning*, 113(7), 4165–4243.
- Alippi, C., Boracchi, G., & Roveri, M. (2016). Hierarchical change-detection tests. *IEEE Transactions on Neural Networks and Learning Systems*, 28(2), 246–258.
- Alippi, C., & Roveri, M. (2008). Just-in-time adaptive classifiers-part I: Detecting nonstationary changes. *IEEE Transactions on Neural Networks*, 19(7), 1145–1153.
- Ang, H.H., Gopalkrishnan, V., & Ng, W.K., et al. (2010). On classifying drifting concepts in p2p networks. In *Joint European conference on machine learning and knowledge discovery in databases*, Springer, (pp 24–39).
- Bach, S.H., & Maloof, M.A. (2008). Paired learners for concept drift. In *2008 Eighth IEEE international conference on data mining*, IEEE, (pp. 23–32).
- Bifet, A., de Francisci Morales, G., & Read, J., et al. (2015). Efficient online evaluation of big data stream classifiers. In *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*, (pp. 59–68).
- Bifet, A., & Gavalda, R. (2007). Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM international conference on data mining*, SIAM, (pp. 443–448).
- Bifet, A., & Gavalda, R. (2009). Adaptive learning from evolving data streams. In *International symposium on intelligent data analysis*, Springer, (pp. 249–260).
- Bifet, A., Holmes, G., & Pfahringer, B. (2010). Leveraging bagging for evolving data streams. In *Machine learning and knowledge discovery in databases: European conference, ECML PKDD 2010, Barcelona, Spain, September 20–24, 2010, Proceedings, Part I* 21, Springer, (pp 135–150).
- Bu, L., Alippi, C., & Zhao, D. (2016). A pdf-free change detection test based on density difference estimation. *IEEE Transactions on Neural Networks and Learning Systems*, 29(2), 324–334.
- Cao, F., Estert, M., & Qian, W., et al. (2006). Density-based clustering over an evolving data stream with noise. In *Proceedings of the 2006 SIAM international conference on data mining*, SIAM, (pp 328–339).
- Chen, C. P., & Zhang, C. Y. (2014). Data-intensive applications, challenges, techniques and technologies: A survey on big data. *Information Sciences*, 275, 314–347.
- Chen, K., Koh, Y.S., & Riddle, P. (2016). Proactive drift detection: Predicting concept drifts in data streams using probabilistic networks. In *2016 International joint conference on neural networks (IJCNN)*, IEEE, (pp 780–787).
- Ditzler, G., Roveri, M., Alippi, C., et al. (2015). Learning in nonstationary environments: A survey. *IEEE Computational Intelligence Magazine*, 10(4), 12–25.
- Domingos, P., & Hulten, G. (2000). Mining high-speed data streams. In *Proceedings of the sixth ACM SIGKDD international conference on Knowledge discovery and data mining*, (pp. 71–80).
- Elwell, R., & Polikar, R. (2011). Incremental learning of concept drift in nonstationary environments. *IEEE Transactions on Neural Networks*, 22(10), 1517–1531.
- Escovedo, T., Koshiyama, A., da Cruz, A. A., et al. (2018). Detecta: Abrupt concept drift detection in non-stationary environments. *Applied Soft Computing*, 62, 119–133.
- Gama, J., Medas, P., & Castillo, G., et al. (2004). Learning with drift detection. In *Advances in artificial intelligence—SBIA 2004: 17th Brazilian symposium on artificial intelligence*, Sao Luis, Maranhao, Brazil, September 29–October 1, 2004. Proceedings 17, Springer, (pp. 286–295).
- Gama, J., Rocha, R., & Medas, P. (2003). Accurate decision trees for mining high-speed data streams. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*, (pp. 523–528).
- Gama, J., Sebastiao, R., & Rodrigues, P.P. (2009). Issues in evaluation of stream learning algorithms. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*, (pp. 329–338).
- Gama, J., Sebastiao, R., & Rodrigues, P. P. (2013). On evaluating stream learning algorithms. *Machine Learning*, 90, 317–346.
- Gama, J., Žliobaitė, I., Bifet, A., et al. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), 1–37.

- Gomes, H. M., Barddal, J. P., Enembreck, F., et al. (2017a). A survey on ensemble learning for data stream classification. *ACM Computing Surveys*, 50(2), 1–36.
- Gomes, H. M., Bifet, A., Read, J., et al. (2017b). Adaptive random forests for evolving data stream classification. *Machine Learning*, 106, 1469–1495.
- Gunasekara, N., Pfahringer, B., & Gomes, H.M., et al. (2024). Recurrent concept drifts on data streams. In *International joint conferences on artificial intelligence organization*.
- Han, M., Chen, Z., Li, M., et al. (2022). A survey of active and passive concept drift handling methods. *Computational Intelligence*, 38(4), 1492–1535.
- Huang, D.T.J., Koh, Y.S., & Dobbie, G., et al. (2014). Detecting volatility shift in data streams. In *2014 IEEE international conference on data mining*. IEEE, (pp 863–868).
- Hulten, G., Spencer, L., & Domingos, P. (2001). Mining time-changing data streams. In *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*, (pp. 97–106).
- Ikononovska, E., Gama, J., & Džeroski, S. (2011). Learning model trees from evolving data streams. *Data Mining and Knowledge Discovery*, 23, 128–168.
- Isah, H., Abughofa, T., Mahfuz, S., et al. (2019). A survey of distributed data stream processing frameworks. *IEEE Access*, 7, 154300–154316.
- Khamassi, I., Sayed-Mouchaweh, M., Hammami, M., et al. (2018). Discussion and review on evolving data streams and concept drift adapting. *Evolving Systems*, 9, 1–23.
- Koh, Y.S., Huang, D.T.J., & Pearce, C., et al. (2018). Volatility drift prediction for transactional data streams. In *2018 IEEE international conference on data mining (ICDM)*, IEEE, (pp 1091–1096).
- Krawczyk, B., Minku, L. L., Gama, J., et al. (2017). Ensemble learning for data stream analysis: A survey. *Information Fusion*, 37, 132–156.
- Li, W., Yang, X., & Liu, W., et al. (2022). Ddg-da: Data distribution generation for predictable concept drift adaptation. In *Proceedings of the AAAI conference on artificial intelligence*, (pp. 4092–4100).
- Lu, J., Liu, A., Dong, F., et al. (2018). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12), 2346–2363.
- Lu, N., Lu, J., Zhang, G., et al. (2016). A concept drift-tolerant case-base editing technique. *Artificial Intelligence*, 230, 108–133.
- Manapragada, C., Webb, G.I., & Salehi, M. (2018). Extremely fast decision tree. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, (pp. 1953–1962).
- Montiel, J., Halford, M., Mastelini, S. M., et al. (2021). River: Machine learning for streaming data in python. *Journal of Machine Learning Research*, 22(110), 1–8.
- Pfahringer, B., Holmes, G., & Kirkby, R. (2007). New options for hoeffding trees. In *AI 2007: Advances in Artificial Intelligence: 20th Australian joint conference on artificial intelligence*, Gold Coast, Australia, December 2-6, 2007. Proceedings 20, Springer, (pp. 90–99).
- Qahtan, A.A., Alharbi, B., & Wang, S., et al. (2015). A pca-based change detection framework for multi-dimensional data streams: Change detection in multidimensional data streams. In *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*, (pp. 935–944).
- Raza, H., Prasad, G., & Li, Y. (2015). Ewma model based shift-detection methods for detecting covariate shifts in non-stationary environments. *Pattern Recognition*, 48(3), 659–669.
- Shaker, A., & Hüllermeier, E. (2015). Recovery analysis for adaptive learning from non-stationary data streams: Experimental design and case study. *Neurocomputing*, 150, 250–264.
- Song, X., Wu, M., & Jermaine, C., et al. (2007). Statistical change detection for multi-dimensional data. In *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*, (pp. 667–676).
- Souza, V. M., dos Reis, D. M., Maletzke, A. G., et al. (2020). Challenges in benchmarking stream learning algorithms with real-world data. *Data Mining and Knowledge Discovery*, 34(6), 1805–1858.
- Wang, H., & Abraham, Z. (2015). Concept drift detection for streaming data. In *2015 International joint conference on neural networks (IJCNN)*, IEEE, (pp. 1–9).
- Wu, O., Koh, Y.S., & Dobbie, G., et al. (2020). Pearl: Probabilistic exact adaptive random forest with lossy counting for data streams. In *Pacific-Asia conference on knowledge discovery and data mining*, Springer, (pp. 17–30).
- Wu, O., Koh, Y.S., & Dobbie, G., et al. (2021). Nacre: Proactive recurrent concept drift detection in data streams. In *2021 International Joint conference on neural networks (IJCNN)*, IEEE, (pp 1–8).
- Yang, Y., Wu, X., & Zhu, X. (2006). Mining in anticipation for concept change: Proactive-reactive prediction in data streams. *Data Mining and Knowledge Discovery*, 13, 261–289.
- Yu, S., & Abraham, Z. (2017). Concept drift detection with hierarchical hypothesis testing. In *Proceedings of the 2017 SIAM international conference on data mining*, SIAM, (pp. 768–776).

- Zeng, X., Zhang, S., Zhong, H., et al. (2024). Pecj: Stream window join on disorder data streams with proactive error compensation. *Proceedings of the ACM on Management of Data*, 2(1), 1–24.
- Zhang, Y., Chu, G., Li, P., et al. (2017). Three-layer concept drifting detection in text data streams. *Neurocomputing*, 260, 393–403.
- Zhao, L., & Shen, Y. (2024). *Proactive model adaptation against concept drift for online time series forecasting*. arXiv preprint [arXiv:2412.08435](https://arxiv.org/abs/2412.08435).

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.